

eRegister Lesotho — v1 Upgrade Toolkit

Operations and reference guide

21 September 2026

Contents

1	eRegister Lesotho — v1 Upgrade Toolkit	5
1.1	The three things you will actually run	5
1.2	Quick start	5
1.3	How to read this guide	6
1.4	Conventions used in this guide	6
2	1. Architecture and layout	7
2.1	1.1 The repository	7
2.2	1.2 Module loading and the curl bash path	7
2.3	1.3 On-disk layout	8
2.4	1.4 The repositories	8
2.4.1	Pinning a ref across all repos	9
2.5	1.5 Privilege	9
2.5.1	Repository ownership	9
2.6	1.6 Prompting	9
3	2. The upgrade: install.sh	11
3.1	2.1 Usage	11
3.2	2.2 What one run does, in order	11
3.2.1	run_migration — the upgrade proper	11
3.2.2	run_post_install — the long tail	12
3.3	2.3 The backup	12
3.3.1	Finding the EMR container	12
3.3.2	An existing dump always wins	13
3.3.3	No old container at all	13
3.4	2.4 Freeze, not destroy	13
3.4.1	Rollback	13
3.5	2.5 What install.sh deliberately does not do	13
3.6	2.6 Re-running after an interruption	14
3.6.1	stage=complete	14
3.6.2	stage=migrated	14
3.7	2.7 Post-install verification	14
3.8	2.8 Worked example	14
3.8.1	Unattended	15
3.8.2	Pinning every repo to one release	15
3.8.3	Installing somewhere other than /var/lib	15
3.9	2.9 Immediately after the upgrade	15
4	3. Reconciling a live site: catch-up.sh	16
4.1	3.1 Usage	16
4.2	3.2 Phase 1 — the self-update	17
4.3	3.3 Phase 2 — the reconcile, step by step	17
4.3.1	Step 1 — Dependency repos	17
4.3.2	Step 2 — Generated helper scripts	18
4.3.3	Step 3 — The scheduled jobs	18
4.3.4	Step 4 — Clinical observation forms	18
4.3.5	Step 5 — Concept dictionary (report only)	18

4.3.6	Step 6 — OpenMRS report definitions	18
4.3.7	Step 7 — The disused identifier source	18
4.3.8	Step 8 — Database backups (report only)	19
4.3.9	Step 9 — Service health	19
4.3.10	Step 10 — Applying the compose files	19
4.3.11	Step 11 — Reloading the EMR	20
4.4	3.4 The report	20
4.5	3.5 Exit status	21
4.6	3.6 Decode-only mode	21
4.7	3.7 Worked examples	21
4.7.1	The normal run	21
4.7.2	Unattended, from the network	21
4.7.3	Deploy new forms, touch nothing else	21
4.7.4	Pick up a new stack release, including moved image tags	21
4.7.5	Reconcile now, reload the EMR later tonight	21
4.7.6	Bring hand-edited repos back onto the release	21
4.8	3.8 As a monitoring check	21
4.9	3.9 The six things catch-up writes	22
5	4. Clinical observation forms	23
5.1	4.1 What gets installed	23
5.2	4.2 The order of operations in a catch-up run	23
5.3	4.3 Retiring the forms a release replaces	23
5.3.1	Why AND retired = 0	24
5.3.2	Why the import state is edited too	24
5.3.3	Before it writes, it shows you	24
5.3.4	Configuration	24
5.3.5	Report rows	24
5.4	4.4 The import	25
5.4.1	Change detection	25
5.4.2	Versioning	25
5.4.3	Concept validation	25
5.4.4	Concurrency	26
5.4.5	Importer flags	26
5.5	4.5 Publishing	26
5.5.1	It re-asserts	26
5.5.2	Publishing without importing	27
5.5.3	Which version gets published	27
5.5.4	Turning it off	27
5.6	4.6 Decoding the deployed form JSON	27
5.6.1	Why repeatedly	27
5.6.2	Return codes	28
5.6.3	Decode on its own	28
5.7	4.7 import-forms.sh — forms on their own	28
5.8	4.8 Running the import by hand	28
5.8.1	Inspecting forms in the database	28
5.9	4.9 Configuration summary	29
6	5. The concept dictionary	30
6.1	5.1 What the import actually does	30
6.2	5.2 Who imports it, and when	30
6.2.1	Why the delay	30
6.2.2	Change detection	31
6.3	5.3 What catch-up.sh reports	31
6.4	5.4 The OCL concept-name fix	31
6.5	5.5 import-concepts.sh	31
6.5.1	Before it writes, it warns	32
6.5.2	Undoing an import	32
6.6	5.6 Choosing which dump to import	32
6.7	5.7 Database connectivity	32

6.8	5.8 Configuration summary	32
7	6. OpenMRS report definitions	34
7.1	6.1 The one import catch-up performs itself	34
7.2	6.2 What one run does	34
7.3	6.3 Which files get imported	34
7.4	6.4 Report rows	35
7.5	6.5 Forcing and undoing	35
7.6	6.6 Configuration summary	35
8	7. Database backups	36
8.1	7.1 The nightly backup	36
	8.1.1 What one run does	36
	8.1.2 Why 01:30	36
	8.1.3 The script is standalone on purpose	36
8.2	7.2 What catch-up checks	37
8.3	7.3 Restoring	37
	8.3.1 Verifying a dump before you trust it	37
8.4	7.4 These dumps do not protect you from losing the machine	37
8.5	7.5 Configuration summary	37
9	8. Scheduled jobs	38
9.1	8.1 systemd or cron	38
9.2	8.2 The auto-pull job	38
9.3	8.3 What catch-up checks	39
9.4	8.4 Inspecting a job	39
	9.4.1 systemd	39
	9.4.2 cron	39
	9.4.3 Either way	39
9.5	8.5 Changing a schedule	39
9.6	8.6 Disabling a job	40
9.7	8.7 The form-import runner in detail	40
10	9. Configuration reference	41
10.1	9.1 Flags by script	41
	10.1.1 install.sh	41
	10.1.2 catch-up.sh	41
	10.1.3 import-concepts.sh	42
	10.1.4 import-forms.sh, ocl-fix.sh	42
10.2	9.2 Core	42
10.3	9.3 The 0.92 source stack	42
10.4	9.4 The v1 stack	42
10.5	9.5 Repository refs	43
10.6	9.6 Clinical forms	43
10.7	9.7 Concept dictionary	43
10.8	9.8 Report definitions	44
10.9	9.9 Identifier source	44
10.10	9.10 Database backups	44
10.11	9.11 Auto-pull	44
10.12	9.12 Catch-up behaviour	45
11	10. Operations runbook	46
11.1	10.1 Upgrade a 0.92 site to v1	46
11.2	10.2 Deploy a new release of the 2026 forms	46
	11.2.1 Verify afterwards	46
11.3	10.3 Change the form-retirement pattern for a new year	47
11.4	10.4 Forms are deployed but show as unpublished	47
	11.4.1 Check what is still in Draft	47
	11.4.2 Keep a particular form unpublished	47
11.5	10.5 A form did not deploy	47

11.6	10.6	Forms render < instead of <	47
11.7	10.7	Pick up a new stack release	48
11.8	10.8	Undo a form retirement	48
11.9	10.9	Undo an identifier-source retirement	48
11.10	10.10	Undo a report-definition import	48
11.11	10.11	Undo a concept-dictionary import	48
11.12	10.12	Restore the database from a nightly dump	49
11.13	10.13	Wire catch-up into monitoring	49
11.14	10.14	Re-point a site at a different branch	49
11.15	10.15	Move the whole install somewhere else	49
11.16	10.16	Re-run an interrupted upgrade	49
11.17	10.17	Check that a release is actually publishable	50
12	11.	Troubleshooting	51
12.1	11.1	curl: (22) The requested URL returned error: 503	51
12.2	11.2	fatal: detected dubious ownership in repository	51
12.3	11.3	A repo row says SKIP ... uncommitted local changes	52
12.4	11.4	openmrsdb:openmrs is not accepting connections right now	52
12.5	11.5	The form import fails or does nothing	52
12.6	11.6	Forms deploy but stay unpublished	52
12.7	11.7	Forms were retired but never came back	53
12.8	11.8	The decode says still escaped after 5 pass(es)	53
12.9	11.9	A nightly job is not producing files	53
12.10	11.10	Changes were pulled but the site does not show them	54
12.11	11.11	The upgrade failed part-way	54
12.12	11.12	install.sh says "already installed" but work is missing	54
12.13	11.13	Getting more detail out of a run	54
12.14	11.14	Where the logs are	54

Chapter 1

eRegister Lesotho — v1 Upgrade Toolkit

This guide documents the scripts that take an eRegister (Bahmni) site from **0.92 to v1**, and that keep a v1 site current afterwards.

eRegister is an EMR built on Bahmni. The toolkit lives in the upgrade-to-v1 repository and is designed to be run either from a checkout or straight off the network with `curl ... | bash`.

1.1 The three things you will actually run

Script	When	What it does
<code>install.sh</code>	Once , to move a site from 0.92 to v1	Backs up the old database, freezes the 0.92 stack, clones the
<code>catch-up.sh</code>	Repeatedly , on a live v1 site	Reconciles the site against the current release: repos, helper
<code>ocl-fix.sh</code>	Once , ~30+ minutes after v1 first starts	Undoes the concept-name changes OCL makes during its fir

Three smaller helpers do one job each, and are covered where that job is documented:

- `import-forms.sh` — the clinical form import, on its own
- `import-concepts.sh` — the concept dictionary, on its own
- `bin/bahmni_form_import.sh` — the form importer itself (installed by the above)

The single most important rule in this guide: once a site is on v1, do **not** re-run `install.sh` to pick up changes. It freezes the stack, restores a backup and restarts everything. Run `catch-up.sh` instead.

1.2 Quick start

Upgrade a 0.92 site:

```
curl -fsSL --retry 8 --retry-max-time 180 \  
↳ https://raw.githubusercontent.com/Lesotho-eRegister-v1/upgrade-to-v1/refs/heads/main/install.sh  
↳ \  
| sudo bash
```

Reconcile a live v1 site:

```
curl -fsSL --retry 8 --retry-max-time 180 \  
↳ https://raw.githubusercontent.com/Lesotho-eRegister-v1/upgrade-to-v1/refs/heads/main/catch-up.sh  
↳ \  
| sudo bash
```

Both are safe to run from a git checkout instead:

```
git clone https://github.com/Lesotho-eRegister-v1/upgrade-to-v1
cd upgrade-to-v1
sudo ./install.sh          # or: sudo ./catch-up.sh
```

1.3 How to read this guide

Chapter	Read it when
1. Architecture and layout	You want to know where things live on disk and how the scripts are put together
2. The upgrade: <code>install.sh</code>	You are moving a site from 0.92 to v1
3. Reconciling a live site: <code>catch-up.sh</code>	You are keeping a v1 site current — the chapter you will re-read most
4. Clinical observation forms	Forms are missing, stale, escaped, or need retiring
5. The concept dictionary	Concepts are missing or a new dictionary has been released
6. OpenMRS report definitions	The Reports app is missing reports
7. Database backups	You need to take, verify or restore a dump
8. Scheduled jobs	A nightly job is not running, or you want to change a schedule
9. Configuration reference	You need the exact name of a flag or environment variable
10. Operations runbook	You want a worked example for a specific task
11. Troubleshooting	Something failed and you need to know what it means

1.4 Conventions used in this guide

- `<base>` is the install base, `/var/lib` unless `--install-dir` says otherwise. Everything the toolkit creates lives under `<base>/v1`.
- Commands shown with `sudo` need root. The scripts detect this themselves and will use `sudo` where required, so running them without it is also fine provided the invoking user can `sudo` without a password prompt mid-run.
- Where a step can be skipped, the flag that skips it is named inline.
- Output samples are real, and are shown without the ANSI colour the scripts actually print.

Chapter 2

1. Architecture and layout

2.1 1.1 The repository

```
upgrade-to-v1/
├── install.sh           the upgrade (0.92 -> v1)
├── catch-up.sh         reconcile a live v1 site
├── import-forms.sh      the form import, standalone
├── import-concepts.sh  the concept dictionary, standalone
├── ocl-fix.sh          the post-startup OCL concept-name fix
├── bin/
│   └── bahmni_form_import.sh  the form importer (installed to /usr/local/bin)
├── lib/
│   ├── core/           config, logging, traps, prompt, cli
│   ├── system/         platform, privilege, deps
│   └── upgrade/        verify, detect, backup, migrate, rollback,
                        postinstall, concepts, reporting, forms, idgen,
                        autopull, dbbackup, oclfix, catchup
└── tests/
    └── check-published.sh  verifies every module is reachable over raw HTTP
```

All logic lives in functions inside `lib/`. The entry scripts hold only their own `main()` and the module list they need.

2.2 1.2 Module loading and the `curl | bash` path

Each entry script declares the modules it needs, in dependency order:

```
REGISTER_MODULES=(
  core/config.sh
  core/logging.sh
  core/traps.sh
  core/prompt.sh
  core/cli.sh
  system/platform.sh
  ...
)
```

If `lib/` sits next to the script, it is sourced from there. If it does not — the `curl | bash` case, where only one file was downloaded — the script fetches the modules itself:

1. **Shallow clone** of the repo. One request, always a self-consistent tree, and it resolves the remote's default branch by itself.
2. **File by file** from `REGISTER_RAW_BASE`, if the clone fails.

It proceeds only once **every** module in the list is present, so a half-published branch fails here with a message naming the missing file, rather than with `curl`'s opaque (56) ... error: 404 somewhere in the middle of a run.

Override the module location with `REGISTER_LIB_DIR` — useful for a system install where `lib/` lives somewhere other than beside the script.

After adding or renaming anything under `lib/` or `bin/`, push it and run `./tests/check-published.sh`. That is what the one-liners fetch.

2.3 1.3 On-disk layout

Everything lives under `<base>/v1`, where `<base>` is `/var/lib` by default:

```
/var/lib/v1/
├── bahmni-docker-ls/           the v1 stack (compose files)
│   └── bahmni-standard/       <- RESTORE_DIR: where compose runs
│       ├── docker-compose.yml
│       ├── restore_bahmni_standard.sh
│       └── run-bahmni.sh
├── standard-config-ls/        Bahmni/clinical app config
├── openmrs-v1-modules/        OpenMRS omods (~246 MB)
├── implementer-interface-release/ the Implementer Interface build
├── clinical-obs-forms/        Form Builder JSON exports
├── dhisconnector_mappings_v1/  DHIS2 connector mappings
├── eregister_concepts_release_v1/ concept dictionary dumps
├── openmrs_reporting_release/  report definitions (serialized_object dump)
├── upgrade-to-v1/             this repo, kept current by catch-up.sh
├── bahmni-backup/
│   ├── openmrssdb_backup.sql   the 0.92 dump the restore reads
│   ├── bahmni_config/          the 0.92 config (pinned, restore-only)
│   ├── concepts-preimport-*.sql taken before each dictionary import
│   └── reporting-preimport-*.sql taken before each report-definition import
├── db-backups/
│   ├── openmrs_<stamp>.sql.gz
│   └── latest.sql.gz           symlink to the newest
├── form-import/               scratch dir for unattended form imports
├── .eregister-upgrade-complete the stage marker (see §2.6)
├── .bahmni_form_import_state.json per-form sha256 + version
├── .eregister_concept_import_state sha256 of the imported dictionary
└── .eregister_reporting_import_state
```

Outside `<base>`:

```
/usr/local/bin/bahmni-form-import.sh   the form importer
/usr/local/bin/eregister-form-import.sh the wrapper cron/systemd runs
/usr/local/bin/eregister-concept-import.sh the concept import runner
/usr/local/bin/eregister-db-backup.sh   the nightly database backup
/usr/local/bin/eregister-autopull.sh    the repo sync
/etc/eregister/form-import.env          EMR credentials (mode 0600)
/var/log/eregister-*.log                one log per job
```

Why the state files live in `<base>/v1` and not inside the clones: the auto-pull job runs `git reset --hard` on those clones. A deployment record stored inside one would be wiped by a routine sync.

2.4 1.4 The repositories

Repo	Default ref	Role	Auto-pulled?
<code>bahmni-docker-ls</code>	<code>Bokang-changes</code>	The v1 stack — compose files	No (pinned)
<code>standard-config-ls</code>	<code>Bokang-changes</code>	Clinical app configuration	Yes
<code>openmrs-v1-modules</code>	<code>main</code>	OpenMRS omods (~246 MB)	Yes
<code>implementer-interface-release</code>	<code>main</code>	Implementer Interface build	Yes
<code>clinical-obs-forms</code>	<code>main</code>	Form Builder JSON exports	Yes
<code>dhisconnector_mappings_v1</code>	<code>master</code>	DHIS2 connector mappings	Yes

Repo	Default ref	Role	Auto-pulled?
eregister_concepts_release_v1	main	Concept dictionary dumps	Yes
openmrs_reporting_release	master	Report definitions	Yes
bahmni_config092	main	The 0.92 config, beside the backup	No (restore-only)
upgrade-to-v1	main	This toolkit	Self-updated by catc

Two deliberate exclusions from the auto-pull job:

- **bahmni-docker-ls** holds the compose files the running stack reads. It must not drift underneath a running instance, so it moves only when a human runs `catch-up.sh`.
- **bahmni_config (0.92)** is historical. It is what the restore reads; it is checked for presence and never updated.

2.4.1 Pinning a ref across all repos

The repos do not share a branch name, so `--target-ref` takes a **comma-separated preference list**, tried in order against each remote. The first ref that exists on that remote wins; the repo's own default is the last resort:

```
sudo ./install.sh --target-ref Bokang-changes,main
```

That resolves to `Bokang-changes` on the stack repos and `main` on the asset repos, in one flag. The ref is resolved against the remote **before** cloning, so a bad ref fails loudly rather than silently landing on the default branch.

A full 40-character SHA is also accepted, and is handled specially: `git clone --branch` rejects a raw SHA, so the repo is cloned in full and the commit is checked out afterwards.

2.5 1.5 Privilege

The scripts detect whether they need elevation and how to get it:

```
[i] Elevation required for /var/lib; will use sudo.
```

Everything that writes under `<base>` or to `/usr/local/bin`, `/etc` or `/var/log` goes through an internal `as_root` helper, so the scripts work both when run as root and when run by a user who can `sudo`.

2.5.1 Repository ownership

Site clones end up owned by whoever created them — root when the installer ran under `sudo`, the operator when it did not. Since git 2.35 a repo whose owner is not the current user is refused outright:

```
fatal: detected dubious ownership in repository at '/var/lib/v1/...'
```

That is how a root-run cron job quietly stops updating a user-owned clone: every command fails, and the caller reads it as "no branch" or "not a repo". Every git call in the toolkit therefore goes through a wrapper:

```
git_here() { git -c safe.directory='*' "$@"; }
```

This relaxes **only** the ownership check, for that one invocation. File permissions still apply exactly as before.

2.6 1.6 Prompting

The scripts are "cautious by default": every step that changes something is confirmed individually.

- An answer that is neither yes nor no is treated as a slip — the prompt asks whether you meant to stop, and repeats itself if you did not.
- Prompts read from `/dev/tty`, never from the script's own `stdin`. This is what makes `curl ... | bash` work: `stdin` is the script itself.
- End-of-input on `/dev/tty` takes the advertised default rather than spinning.
- `--yes` (or `EREGISTER_ASSUME_YES=1`) answers yes to everything. Required for unattended runs, and worth thinking about carefully — see §3.11.

Passwords are never echoed, never passed on a command line where another user could see them in ps, and never hard-coded:

- The 0.92 database password is prompted, or read from EREGISTER_DB_PASS.
- The EMR password is prompted, or read from EREGISTER_BAHMNI_PASS, and is stored 0600 in /etc/eregister/form-import.env for the unattended runs.
- Inside containers, MYSQL_PWD or the container's own MYSQL_ROOT_PASSWORD is used, so the password never lands on a process list.

Chapter 3

2. The upgrade: `install.sh`

`install.sh` works as both a fresh installer and an in-place upgrader from 0.92 to v1. It is the only script in the toolkit that stops the old stack and restores a database.

Run it **once** per site. To pick up later changes to these scripts on a site that is already on v1, run `catch-up.sh` — see [chapter 3](#).

3.1 2.1 Usage

```
./install.sh [--yes] [--force] [--install-dir DIR] [--target-ref REF[,REF...]]
              [--no-concepts] [--no-db-backup] [--no-color] [--help]
```

Flag	Meaning
<code>-y, --yes</code>	Non-interactive; assume yes at every prompt. Also <code>EREREGISTER_ASSUME_YES=1</code>
<code>--force</code>	Redo the whole upgrade even when the marker says it finished
<code>--install-dir DIR</code>	Install base (default <code>/var/lib</code>)
<code>--target-ref REF[,REF...]</code>	Git ref preference list, tried against every repo — see §1.4
<code>--no-concepts</code>	Do not install the concept-dictionary job at all
<code>--no-db-backup</code>	Do not install the daily database backup. The site is then left with no routine backup
<code>--no-forms</code>	Accepted but inert here — this script no longer touches forms
<code>--no-color</code>	Disable ANSI colours

3.2 2.2 What one run does, in order

```
main()
├─ load_modules           source lib/ (or fetch it — see §1.2)
├─ parse_args / banner
├─ detect_platform        OS, arch
├─ detect_pkg_mgr         apt-get, yum, ...
├─ resolve_config         finalise every <base>-derived path
├─ detect_privilege       root, or sudo
├─ print_config           the "Resolved configuration" block
├─ resolve_install_stage  the idempotency guard (§2.6)
├─ run_migration          ← the upgrade proper
├─ run_post_install       ← the scheduled jobs
├─ mark_stage complete
└─ next_steps
```

3.2.1 `run_migration` — the upgrade proper

#	Step	Notes
1	Go/no-go	Begin the upgrade 0.92 -> v1? Every step below is <i>also</i> confirmed individually
2	Dependencies	Check for, and install if missing: git, docker, ...
3	Workspace and folders	mktemp -d plus <base>/v1 and <base>/v1/bahmni-backup
4	Backup	Dump openmrs out of the running 0.92 EMR container
5	Freeze the old stack	docker compose stop — rollback is armed from here
6	Fetch repos	Clone/update every v1 repo plus the 0.92 config
7	Restore	restore_bahmni_standard.sh <backup dir>
8	Start v1	run-bahmni.sh, falling back to docker compose up -d
9	Verify and mark	Post-install checks, then stage=migrated

3.2.2 run_post_install — the long tail

Every step here is **advisory**: the migration is already finalised, so a failure warns and names the standalone script that redoes it. None of them can abort the run or trigger a rollback.

#	Step	Notes
1	Daily database backup	First, deliberately — see below
2	Concept-dictionary job	Installed with one delayed first run, ~3h out
3	<i>(clinical forms)</i>	Deliberately absent — see §2.5
4	Auto-pull job	Offered, then installed

The database backup is installed **first** on purpose. Everything else in this function installs something that will later write to the openmrs database on a schedule — the concept dictionary replaces whole tables, the form import creates form versions. The job that lets a site undo any of that should be in place before those jobs are, not after.

3.3 2.3 The backup

This is the dump the restore reads. It is taken **before anything is changed**, directly out of the running 0.92 EMR container:

```
docker exec -e "MYSQL_PWD=<password>" <emr-container> \
  mysqldump --single-transaction --routines --triggers --events \
    --hex-blob --default-character-set=utf8mb4 \
    --databases -u root openmrs \
  > /var/lib/v1/bahmni-backup/openmrssdb_backup.sql
```

Each flag earns its place:

Flag	Why
--single-transaction	Consistent InnoDB snapshot without locking writers
--routines	Stored procedures and functions
--triggers	Table triggers
--events	Scheduled events
--databases	Emits CREATE DATABASE / USE, so the DB is recreated
--hex-blob	Binary-safe encoding of BLOB/BINARY columns
--default-character-set=utf8mb4	No truncation or corruption of unicode text

MYSQL_PWD keeps the password off the container's process list.

3.3.1 Finding the EMR container

The configured name is tried first, then a known alternate, each matched with hyphen and underscore treated as interchangeable — so compose v1 (bahmni_docker_emr_service_1), compose v2 (bahmni_docker-emr-service-1) and mixed variants all resolve to the same configured name.

If neither is running, you are asked for the name, **in a loop**: a mistyped name used to cost the pre-upgrade backup outright. A blank line still means "skip", and a name that does not resolve gets a second chance plus a listing of the running containers to copy from.

3.3.2 An existing dump always wins

A dump already sitting in bahmni-backup is **reused, not retaken**. It is the 0.92 data the restore needs, and once an earlier run has frozen the old stack it cannot be retaken at all.

```
[✓] Reusing the backup already in /var/lib/v1/bahmni-backup:
[✓] /var/lib/v1/bahmni-backup/openmrsdb_backup.sql (299M)
[i] The restore below loads THIS file. To retake it instead, re-run with
    --force while the 0.92 EMR container is running.
```

--force reloads the database from that file, replacing anything entered since it was made.
On a site that has been live on v1 for a while, that is data loss. This is why catch-up.sh exists.

3.3.3 No old container at all

If nothing is running and no dump exists, the run continues as a **fresh install** with nothing to migrate. Downstream steps notice: the restore is still attempted but never fatal, and every post-install check is demoted from an error to a warning.

3.4 2.4 Freeze, not destroy

```
cd ~/bahmni_docker && docker compose stop
```

stop, not down. The containers are halted but **not removed**, so volumes and networks stay intact and a rollback is a fast start with no re-create.

3.4.1 Rollback

From the freeze onwards, an error trap is armed. If the run fails or is aborted before the migration is finalised:

```
[△] Upgrade failed – initiating rollback to eRegister Lesotho 0.92.
[i] Restarting the frozen old stack (/home/kgatman/bahmni_docker)...
[✓] Old stack restarted.
[△] Your database backup is preserved at: /var/lib/v1/bahmni-backup/openmrsdb_backup.sql
[✗] Rollback complete. The v1 upgrade was NOT applied.
```

The backup is always preserved. Rollback disarms once post_verify succeeds.

3.5 2.5 What install.sh deliberately does not do

Three imports are cloned but not performed here. The reason is the same in each case, and it is worth understanding because it shapes the whole toolkit:

When install.sh finishes, the stack has only just been started. openmrsdb answers early, but the OpenMRS instance behind it needs 30+ minutes — hours on site hardware — before it is usable.

Not done here	Why
Concept dictionary	The import replaces the concept_*/drug* tables. Asking an operator to authorise that at the least
Clinical forms	They deploy over the EMR's REST API , which is not answering yet. An import here either blocks
Report definitions	Same reason as the dictionary: the database is minutes old

For the forms this is absolute: install.sh installs **no** form importer, **no** credentials file and **no** timer. Until catch-up.sh or import-forms.sh has been run, nothing on the host is importing forms.

3.6 2.6 Re-running after an interruption

<base>/v1/.eregister-upgrade-complete records **how far** the last run got, not merely that one happened:

Marker	Meaning	A re-run will...
(absent)	Nothing has run	Do the whole upgrade
stage=migrated	Stack migrated, verified and started, but the post-install steps did not finish	Redo the upgrade from
stage=complete	The whole run finished	Short-circuit: "already"

This distinction is the whole point of the file. The post-install steps load a multi-hundred-MB dump and talk to an EMR that takes 30+ minutes to boot, so they are easy to interrupt with a Ctrl-C or a dropped ssh session. When the marker was a bare touch taken at verification time, an interrupt anywhere in that window left behind a marker saying "installed" — and every later run short-circuited on it, printing a summary of work it had not done.

A marker written by an older installer carries no stage. What is still outstanding is then inferred from what those steps leave on disk (the concept-import runner, or its state file).

3.6.1 stage=complete

```
[✓] eRegister Lesotho v1 is already installed (/var/lib/v1/.eregister-upgrade-complete).
    Nothing to do.
[i] To pick up later changes to these scripts on a live site, run ./catch-up.sh.
[i] To redo the whole upgrade from the backup in /var/lib/v1/bahmni-backup,
    re-run with --force.
```

3.6.2 stage=migrated

```
[△] A previous run migrated the stack but stopped before the post-install
[△] steps finished (/var/lib/v1/.eregister-upgrade-complete records stage=migrated).
[△] This run redoes the upgrade from the top. The backup already in
[△] /var/lib/v1/bahmni-backup is REUSED, not retaken — and the restore reloads
[△] 'openmrs' from it, replacing anything entered since it was made.
```

3.7 2.7 Post-install verification

Before recording stage=migrated, four things are checked:

```
[ -s <base>/v1/bahmni-backup/openmrsdb_backup.sql ] the dump exists and is non-empty
[ -d <base>/v1/bahmni-docker-ls/.git ]           the stack repo is a checkout
[ -d <base>/v1/standard-config-ls/.git ]         the config repo is a checkout
[ -d <base>/v1/bahmni-backup/bahmni_config/.git ] the 0.92 config is a checkout
```

On a fresh install (no backup taken), each of these is demoted to a warning so the upgrade can still finalise.

3.8 2.8 Worked example

A full upgrade on a host that already holds a 0.92 dump:

```
curl -fsSL --retry 8 --retry-max-time 180 \
```

```
↳ https://raw.githubusercontent.com/Lesotho-eRegister-v1/upgrade-to-v1/refs/heads/main/install.sh
↳ | bash
```

```
lib/ not found locally — fetching the modules ...
```

```
Modules obtained by cloning https://github.com/Lesotho-eRegister-v1/upgrade-to-v1.
```

```

███ eRegister Lesotho — Upgrade to v1
[..] Initializing migration.....
```

```
|      [??] Checking prerequisites.....  
|      [**] Doing the magic.....  
|
```

[i] Elevation required for /var/lib; will use sudo.

==> Resolved configuration

```
App           : eRegister Lesotho  
Current ver   : none  
Target ver    : v1  
Ref override  : (none - using the per-repo defaults below)  
Default refs  : docker=Bokang-changes config=Bokang-changes 092=main  
OS / Arch     : linux / amd64  
Pkg manager   : apt-get  
Install base  : /var/lib  
v1 dir        : /var/lib/v1  
Concept dict  : /var/lib/v1/eregister_concepts_release_v1/omrs_concept_dictionary*.sql  
↳ (newest) -> openmrssdb:openmrs  
Concept job   : first run ~3h after install, then daily 30 4 * * *  
Report defs   : /var/lib/v1/openmrs_reporting_release (cloned here; imported by  
↳ ./catch-up.sh)  
Clinical forms : /var/lib/v1/clinical-obs-forms (cloned here; imported by ./catch-up.sh)  
DB backup     : openmrssdb:openmrs -> /var/lib/v1/db-backups (daily: 30 1 * * *, keep 14)  
Old stack     : /home/kgatman/bahmni_docker  
EMR container  : bahmni_docker-emr-service-1  
Privilege     : sudo  
Non-interactive: no
```

[△] Cautious mode: you will be asked to confirm EVERY step before it runs.

Begin the upgrade 0.92 -> v1? [y/N]: y

Next step: Check for, and install if missing, required dependencies (git, docker, ...) -

↳ proceed? [y/N]: y

[✓] All dependencies present.

...

3.8.1 Unattended

```
sudo EREGISTER_DB_PASS='<0.92 db password>' ./install.sh --yes
```

3.8.2 Pinning every repo to one release

```
sudo ./install.sh --yes --target-ref v1.4.0,Bokang-changes,main
```

3.8.3 Installing somewhere other than /var/lib

```
sudo ./install.sh --install-dir /srv  
# everything then lives under /srv/v1
```

3.9 2.9 Immediately after the upgrade

The stack is up but the EMR is still booting. In order:

1. **Wait 30+ minutes.** Hours on site hardware. Nothing below works until the EMR answers.
2. **Run ocl-fix.sh** once the OCL import has finished — see [chapter 5](#).
3. **Run catch-up.sh.** This is what deploys the clinical forms and the report definitions. Until it has run, neither is on the site.

Chapter 4

3. Reconciling a live site: `catch-up.sh`

`catch-up.sh` re-checks everything `install.sh` is meant to have set up, and redoes **only** what is missing or out of date.

It exists because sites installed from an earlier version of these scripts never got the steps added since — and re-running `install.sh` on them is the wrong tool, because that freezes the stack, restores a backup and restarts everything.

It also **owns** two steps outright: the clinical form import and the report definition import happen here and nowhere else.

The run ends with a single table: what was already OK, what it redid, what it deliberately left alone, and what still needs a human. **Exit status is 0 only when there are no gaps**, so it doubles as a monitoring check.

4.1 3.1 Usage

```
./catch-up.sh [--decode]
./catch-up.sh [--yes] [--no-stack] [--no-forms] [--no-retire-forms]
               [--no-publish] [--no-decode] [--no-concepts] [--no-reporting]
               [--no-idgen] [--no-db-backup] [--no-compose-up] [--pull-images]
               [--no-recreate] [--force-repos]
               [--install-dir DIR] [--no-color] [--help]
```

Flag	Effect
<code>-y, --yes</code>	Non-interactive; assume yes at every prompt
<code>--decode / --decode-only</code>	Run only the form-JSON decode, then stop. No repo updated, nothing imported, no h
<code>--no-stack</code>	Do not fast-forward <code>bahmni-docker-ls</code>
<code>--no-forms</code>	Leave the form import and its schedule alone. Implies <code>--no-decode</code> , <code>--no-retire-forms</code>
<code>--no-retire-forms</code>	Import the new forms but leave the ones they replace live
<code>--no-publish</code>	Leave the deployed forms in Draft — they are then not offered in the clinical app
<code>--no-decode</code>	Import the forms but leave the HTML entities in what the EMR wrote
<code>--no-concepts</code>	Leave the dictionary alone: no DB probe, and the daily job is neither installed nor refresh
<code>--no-reporting</code>	Clone and fast-forward the reporting repo, but do not import it
<code>--no-idgen</code>	Do not retire the disused identifier source
<code>--no-db-backup</code>	Leave the nightly backup alone — not installed, refreshed or reported on
<code>--no-compose-up</code>	Do not apply the compose files to the stack
<code>--pull-images</code>	Add <code>--pull</code> always when applying them
<code>--no-recreate</code>	Do not recreate the EMR service at the end
<code>--force-repos</code>	Bring off-release repos back onto their pinned ref, discarding local changes
<code>--install-dir DIR</code>	Install base (default <code>/var/lib</code>)

`--decode` and `--no-decode` contradict each other. Passing both is an error, because picking one silently would leave you working out why your run did nothing.

4.2 3.2 Phase 1 — the self-update

Before any module is sourced, `catch-up.sh` updates **itself**, so a site always reconciles against the latest version of these scripts. This phase may use only shell built-ins, `git` and `curl`.

Two cases:

- **Running from a git checkout** — fast-forward it in place. A checkout with uncommitted local changes is never discarded; it is reported and left alone.
- **Piped from curl, or a stray copy** — clone or update the managed checkout at `<base>/v1/upgrade-to-v1` and re-exec from there.

Either way it execs the up-to-date `catch-up.sh` exactly once, guarded by `EREGISTER_CATCHUP_REEXEC`, so the rest of the run uses the new modules. The outcome is handed to phase 2 in the environment and becomes the first row of the report:

```
◦ FIXED self      upgrade-to-v1    a1b2c3d -> e4f5a6b (/var/lib/v1/upgrade-to-v1)
```

If `git` is missing, or the clone fails, the run continues with the copy that is running and records a GAP.

4.3 3.3 Phase 2 — the reconcile, step by step

```
catch_up()
├─ 1. catchup_repos           dependency repos
├─ 2. catchup_helper_scripts generated helpers
├─ 3. catchup_schedules      the four scheduled jobs
├─ 4. catchup_forms          retire → import → decode
├─ 5. catchup_concepts       REPORT ONLY
├─ 6. catchup_reporting      report definitions (imported here)
├─ 7. catchup_idgen          retire the disused identifier source
├─ 8. catchup_db_backups     REPORT ONLY
├─ 9. catchup_services       health, as found
├─ 10. catchup_stack_up      docker compose up -d
├─ 11. catchup_recreate_emr  reload the EMR
├─ 12. catchup_report        the table + verdict
```

4.3.1 Step 1 — Dependency repos

Every repo the installer clones is fast-forwarded onto its pinned ref, plus the site's own `upgrade-to-v1` checkout. Each gets a report row.

Repos are classified, and the class decides the behaviour:

Class	Repos
asset	standard-config-ls, openmrs-v1-modules, implementer-interface-release, clinical-obs-forms
stack	bahmni-docker-ls
pinned	bahmni_config (0.92)
self	upgrade-to-v1

A repo is reported and left completely alone — never reset — when it:

- has uncommitted local changes, or
- sits on a detached HEAD, or
- tracks a branch other than the one this release pins.

```
– SKIP repo      standard-config-ls  uncommitted local changes – left untouched;
◁ --force-repos to reset it onto Bokang-changes
```

Sites do hand-edit config, and silently discarding that would be the one destructive thing this script could plausibly do. `--force-repos` opts in.

A clone pointing at a different remote than the release configures is flagged in the row, and re-pointed **only** once it is otherwise on-release — re-pointing a repo that is also off-ref would leave it tracking a branch its new origin may not even have.

4.3.2 Step 2 — Generated helper scripts

`eregister-autopull.sh`, `eregister-db-backup.sh`, `bahmni-form-import.sh`, `eregister-form-import.sh` are rewritten from the release that was just pulled. They are generated files, so rewriting unconditionally is how a site picks up fixes to them, and keeps the result identical to a fresh install.

This is also where the form importer is **installed** rather than merely refreshed: `install.sh` does no form work at all, so on a site that has never run catch-up none of those three exist yet.

The EMR password in `/etc/eregister/form-import.env` is tested against the EMR:

- a working one is left alone,
- a missing or **rejected** one is replaced after prompting (and checked against the EMR before it is saved),
- while the EMR is still booting nothing can be checked, so an existing file is kept as-is and checked again just before the import runs.

4.3.3 Step 3 — The scheduled jobs

All four jobs (`eregister-db-backup`, `eregister-autopull`, `eregister-form-import`, `eregister-concept-import`) are checked, and whichever is absent is installed — as a systemd timer, or an `/etc/cron.d` entry where systemd is missing. Hosts with neither get the exact cron line to add by hand.

This is how a site installed before the backup job existed gets one.

4.3.4 Step 4 — Clinical observation forms

Three parts, each with its own report row. Covered in full in [chapter 4](#):

1. **Retire** the forms this release replaces (`--no-retire-forms` skips)
2. **Import** the forms whose content changed, and **publish** them (`--no-forms` skips the import, `--no-publish` skips just the publishing)
3. **Decode** the HTML entities in the JSON the EMR wrote (`--no-decode` skips)

Publishing is what makes a deployed form actually appear in the clinical app — the Implementer Interface's Import button leaves it in Draft. It re-asserts on every run, so forms that were deployed as drafts before this existed are published by the next ordinary catch-up.

The decode is deliberately **not** conditional on the import: what is being decoded is JSON the EMR already holds, so a run that deployed nothing can still face a folder full of `<`; from an earlier release.

4.3.5 Step 5 — Concept dictionary (report only)

Catch-up **never** imports the dictionary. That replaces the `concept_*/drug*` tables, which is the daily concept job's business — or yours via `./import-concepts.sh`.

It reports which dump is on disk, whether it is the one actually imported (comparing sha256 against the import marker), and the live concept row count. The severity follows from that:

- a newer dump **with** the job scheduled → OK (pending tonight)
- the same dump with **no** job scheduled → GAP, because then nothing will ever load it

4.3.6 Step 6 — OpenMRS report definitions

Unlike the dictionary, this one **is** imported here. See [chapter 6](#). It replaces a single table (`serialized_object`), backs that table up first, and does nothing at all when the dump in the clone is already the one in the database.

4.3.7 Step 7 — The disused identifier source

Marks one row of `idgen_identifier_source` retired in the `openmrs` database, so it stops being offered for new identifiers:

```

UPDATE idgen_identifier_source
SET retired = 1, retired_by = 1, date_retired = NOW(),
    retire_reason = 'No longer in use'
WHERE id = 14 AND retired = 0;

```

The row stays, and so does every identifier it has already issued. The run prints the single statement that undoes it.

AND retired = 0 matters: without it every run rewrites date_retired on a row retired months ago, so the site loses the date it was actually retired and the report claims a change it did not make. With it, the second and every later run match zero rows and the step is a true no-op — **the database is its own state file**.

The flip side is deliberate: this **re-asserts**. An operator who un-retires that source by hand will find the next catch-up retiring it again. Pass --no-idgen on a site where it is wanted.

Ids are per-site auto-increments, so the step reads the row first and the report names what it retired. An id that is not there is a GAP, not a quiet pass:

```

x GAP   idgen      source 14          no source with id 14 in openmrs - nothing retired (set
  ↪  EREGISTER_IDGEN_RETIRE_ID, or --no-idgen)

```

4.3.8 Step 8 — Database backups (report only)

Step 3 says a timer *exists*. This row says the timer is *producing files*, which is a different question and the one that matters: a backup job whose openmrsdb service has been unreachable since a compose rename fails silently every night, and the only visible symptom is that the newest dump stops moving.

A newest dump over 36h old is a GAP, as is any leftover .part file.

4.3.9 Step 9 — Service health

docker compose ps per service, plus HTTP probes of the OpenMRS REST API and the Bahmni UI. This is the site **as found** — probed before anything below reloads it.

4.3.10 Step 10 — Applying the compose files

```
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard && docker compose up -d
```

Step 1 fast-forwarded bahmni-docker-ls, which **is** the compose files this command reads. Without this step, pulling that repo changes files nothing ever reads, and a site can sit for months on a stack definition it already has on disk.

The EMR reload in step 11 is **not** a substitute: it names one service, so a new service, a changed image tag, a new port or a changed environment block anywhere else in the file is invisible to it.

up -d is a **reconcile, not a restart**. A service whose resolved definition did not change is left running exactly as it is; only containers whose definition moved are recreated, plus anything missing or stopped. On a site whose bahmni-docker-ls did not move, this is a no-op.

That does include the database service, if the compose file changed it. Named volumes — where the patient data lives — are never touched by up -d, but it can mean a few seconds of database downtime. Hence the confirm, and --no-compose-up.

Images. By default this deploys the images the host already has, pulling only one it has never seen. A release that moves a tag *in place* (:latest and friends) therefore needs --pull-images, which adds --pull always. It is off by default because it turns a fast local reconcile into a download over whatever link the site has.

The row is derived by diffing docker compose ps -a -q before and after, rather than by parsing compose's progress output — that wording is a UI detail that changes between compose releases, while an id that is gone is unambiguously a container that was replaced.

4.3.11 Step 11 — Reloading the EMR

```
docker compose up -d --force-recreate --renew-anon-volumes openmrs
```

The steps above update files on disk — standard-config, the omods, the implementer interface, the forms. A long-running openmrs container goes on serving what it read at boot, and anything seeded into an **anonymous** volume when the container was first created keeps the old content even across a plain restart. `--force-recreate` replaces the container; `--renew-anon-volumes` throws its anonymous volumes away so they are re-seeded.

This takes the EMR down for its usual 30+ minute boot. Named volumes and the separate openmrsdb service (the patient data) are **not** touched, so this is not a data-loss operation — but anything a site hand-placed *inside* the running EMR container, rather than in its config repo, is gone.

Hence the confirm, `--no-recreate`, and `EREGISTER_CATCHUP_RECREATE=0`.

4.4 3.4 The report

```
=====
eRegister v1 - catch-up report (2026-09-21 14:22:05 SAST)
=====
```

⊖ FIXED self	upgrade-to-v1	a1b2c3d -> e4f5a6b (/var/lib/v1/upgrade-to-v1)
✓ OK repo	standard-config-ls	current (Bokang-changes @ 7f3c1a2)
⊖ FIXED repo	clinical-obs-forms	main 4d9e0b1 -> 8c2a7f5
- SKIP repo	bahmni-docker-ls	left at 22b9e10 (--no-stack)
✓ OK script	eregister-autopull.sh	present, refreshed from this release
✓ OK cron	eregister-form-import	systemd timer active (next: Mon 03:30)
⊖ FIXED forms	retire	12 form(s) retired (reason: deploying latest forms ↳ ...)
⊖ FIXED forms	import	imported 12/48 form(s), 30 unchanged, 12 published, ↳ 0 failed
✓ OK forms	decode	nothing escaped in ↳ openmrs:/home/bahmni/clinical_forms
✓ OK concepts	dump	omrs_concept_dictionary_20260804.sql is the one ↳ imported
✓ OK reporting	import	already current (sha256 matches)
✓ OK idgen	source 14	'Old ART Number' already retired
✗ GAP backup	newest dump	newest dump is 51h old - the nightly job is not ↳ producing files
✓ OK service	openmrs	running
⊖ FIXED stack	compose up	3 container(s) created or recreated from ↳ /var/lib/v1/...
⊖ FIXED reload	openmrs	recreated with renewed anonymous volumes - booting ↳ now

[i] OK = already current · FIXED = redone by this run · SKIP = left alone · GAP = needs
↳ attention

[△] 1 item(s) need attention - see the ✗ rows above.

Status	Symbol	Meaning
OK	✓	Already current — this run changed nothing
FIXED	⊖	Redone by this run
SKIP	-	Deliberately left alone (a flag, or you declined)
GAP	✗	Needs a human

The footer also prints the undo statements for everything the run wrote, and the commands to run each job by hand.

4.5 3.5 Exit status

```
sudo ./catch-up.sh --yes; echo $?
```

0 when there are no GAP rows; non-zero otherwise. This is what makes it usable as a monitoring check.

4.6 3.6 Decode-only mode

The fast path for a site whose forms are already deployed and only need the entity clean-up:

```
sudo ./catch-up.sh --decode
```

No repo is updated, nothing is imported or scheduled, no health probe runs and the EMR is left running — so it costs seconds instead of the usual run. The self-update row is kept (it already happened in phase 1), and the verdict is scoped honestly:

```
[✓] Decode complete: nothing left escaped. Nothing else was checked.
```

4.7 3.7 Worked examples

4.7.1 The normal run

```
cd /var/lib/v1/upgrade-to-v1 && sudo ./catch-up.sh
```

4.7.2 Unattended, from the network

```
curl -fsSL --retry 8 --retry-max-time 180 \
```

```
↪ https://raw.githubusercontent.com/Lesotho-eRegister-v1/upgrade-to-v1/refs/heads/main/catch-up.sh
↪ \
  | sudo EREGISTER_BAHMNI_PASS='<superman password>' bash -s -- --yes
```

4.7.3 Deploy new forms, touch nothing else

```
sudo ./catch-up.sh --no-stack --no-concepts --no-reporting --no-idgen \
                  --no-db-backup --no-compose-up
```

4.7.4 Pick up a new stack release, including moved image tags

```
sudo ./catch-up.sh --yes --pull-images
```

4.7.5 Reconcile now, reload the EMR later tonight

```
sudo ./catch-up.sh --no-compose-up --no-recreate
# then, in the maintenance window:
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard
sudo docker compose up -d
sudo docker compose up -d --force-recreate --renew-anon-volumes openmrs
```

4.7.6 Bring hand-edited repos back onto the release

```
sudo ./catch-up.sh --force-repos      # DISCARDS local changes in those clones
```

4.8 3.8 As a monitoring check

```
30 6 * * 1 /var/lib/v1/upgrade-to-v1/catch-up.sh --yes --no-forms --no-idgen \
          --no-compose-up --no-recreate >> /var/log/eregister-catchup.log 2>&1
```

Note what that line switches off. `--yes` answers every confirmation with "yes", so a scheduled run would otherwise retire and re-import forms, retire the identifier source, apply the compose files to the stack and reload the EMR unattended, week after week.

`--no-compose-up` matters most here: without it an unattended run would recreate whichever containers a freshly-pulled `bahmni-docker-ls` changed, at 06:30 on a Monday, with nobody watching.

A monitoring check should report the site, not change it. Keep the skips, and do the writing runs by hand.

4.9 3.9 The six things catch-up writes

Everything else in a run is read-only. These are the exceptions, each individually confirmed and each with a flag that skips it:

Write	Reversible?	Skip with
Retire the forms this release replaces	Yes — one UPDATE, printed by the run	<code>--no-retire-forms</code>
Decode HTML entities in the EMR's form JSON	Textual, idempotent	<code>--no-decode</code>
Import the report definitions	Yes — a pre-import dump is taken first	<code>--no-reporting</code>
Retire the disused identifier source	Yes — one UPDATE, printed by the run	<code>--no-idgen</code>
<code>docker compose up -d</code> on the whole stack	Re-runnable; named volumes untouched	<code>--no-compose-up</code>
Recreate the EMR service	Re-runnable; 30+ min downtime	<code>--no-recreate</code>

Each of the two database writes earns its place the same way: one table, no patient data, reversible in a single statement, and **nothing else on the site is ever going to do it**. Reporting them as gaps would leave those gaps open forever.

Chapter 5

4. Clinical observation forms

The `clinical-obs-forms` repo holds Bahmni **Form Builder JSON exports**. `bin/bahmni_form_import.sh` replays what the Implementer Interface's "Import" button does — concept UUID fix-up, `POST /form`, save body, save translations — over the EMR's REST API, so those exports become live forms without anyone clicking through the UI. It then does the one thing that button does **not**: it publishes the form (§4.5).

install.sh does none of this. The forms deploy over the REST API, and at the moment the upgrade finishes the EMR needs 30+ minutes before it answers one. The entire step — importer, credentials, runner, schedule and the first import — belongs to `catch-up.sh`, with `import-forms.sh` as the forms-only entry point.

5.1 4.1 What gets installed

Path	What
<code>/usr/local/bin/bahmni-form-import.sh</code>	The importer itself
<code>/usr/local/bin/eregister-form-import.sh</code>	The wrapper cron/systemd runs
<code>/etc/eregister/form-import.env</code>	Credentials, mode <code>0600</code> , root-owned
<code>/var/lib/v1/.bahmni_form_import_state.json</code>	Per-form sha256 + version
<code>/var/lib/v1/form-import/</code>	Scratch dir for <code>importErrors.txt</code>
<code>/var/log/eregister-form-import.log</code>	The log
<code>eregister-form-import.timer</code> or <code>/etc/cron.d/eregister-form-import</code>	Daily at 03:30

The runner is a **bare-environment wrapper**: no `lib/`, no `PATH` assumptions, its own log. It sources the `env` file, optionally refreshes the clone, then runs the importer over the whole folder.

5.2 4.2 The order of operations in a catch-up run

```
_cu_forms_import
├─ is the runner installed and are there credentials? → GAP if not
├─ "Run the form import now?" → SKIP if declined
├─ re-check the EMR password → GAP if rejected
├─ _forms_retire_stale ← retire the outgoing set
├─ run_form_import ← deploy the incoming set, and publish it
└─ _cu_forms_decode ← always, even if nothing was imported
```

The retirement sits **inside** the import step, after the confirmation and the credential check. That is deliberate: retiring the 2026 forms and then not deploying the new ones would leave the site with nothing to fill in. The two halves must not come apart.

5.3 4.3 Retiring the forms a release replaces

Straight before the import, every **live** form whose name matches the configured pattern is marked retired:


```
UPDATE form SET retired = 1, retired_by = 1, date_retired = NOW(),
  retire_reason = 'deploying latest forms with the latest changes - kgatman'
WHERE name LIKE '%2026%' AND retired = 0;
```

This is how the year's outgoing set stops being offered at the same moment the incoming one lands.

OpenMRS retires rather than deletes. The rows stay, and every observation ever recorded against those forms keeps resolving. One UPDATE undoes it, and the run prints it:

```
[i] Undo it with:
[i]   UPDATE form SET retired = 0, retired_by = NULL, date_retired = NULL,
[i]       retire_reason = NULL WHERE retire_reason = 'deploying latest forms with the
  ↳ latest changes - kgatman';
```

5.3.1 Why AND retired = 0

It is the script's own addition to the statement, for the same reason `idgen.sh` carries it: without it, every run rewrites `date_retired` on rows retired months ago, so the site loses the date they were actually retired and the report claims a change it did not make.

5.3.2 Why the import state is edited too

The importer deploys a form only when its **file** changed — and retiring a form does not change its file. Without intervention the very forms just retired would be skipped as "unchanged", and the site would be left with **no live copy of them at all**.

So the retirement blanks the recorded sha256 for exactly the retired form names. It deliberately **keeps the recorded version**: the importer deploys `max(state version, server version) + 1`, and a retired form is not in the server's answer, so dropping the version could make it redeploy a number that already exists on a retired row.

```
[i] Cleared the recorded hash of the retired forms so the import redeploys them.
```

This is best-effort. If `jq` is missing or the state file cannot be rewritten, you get a warning naming the manual fallback:

```
sudo /usr/local/bin/bahmni-form-import.sh --force -r /var/lib/v1/clinical-obs-forms
```

5.3.3 Before it writes, it shows you

```
[i] 12 live form(s) in 'openmrs' match name LIKE '%2026%':
  • ANC Intake 2026 (Nurse's)
  • HIV Treatment and Care Follow Up 2026
```

```
...
```

```
[Δ] Retiring them stops them being offered. The rows are kept and the
```

```
[Δ] observations already recorded against them are NOT touched – this is reversible.
```

```
Retire these 12 form(s) before importing? [y/N]:
```

5.3.4 Configuration

Variable	Default	Meaning
EREGISTER_FORM_RETIRE	1	0 (off)
EREGISTER_FORM_RETIRE_NAME_LIKE	%2026%	SQL LIKE pattern
EREGISTER_FORM_RETIRE_REASON	deploying latest forms with the latest changes - kgatman	Write reason for retirement
EREGISTER_FORM_RETIRE_BY	1	User ID

Values are escaped before being interpolated into the SQL: backslashes first (MySQL's own escape character), then single quotes.

5.3.5 Report rows

Status	Row	When
FIXED	N form(s) retired (reason: ...)	Rows were retired
OK	no live form matches name LIKE '%2026%' in openmrs	Nothing to do
SKIP	left alone (--no-retire-forms)/declined – N form(s) left live	Disabled or declined
GAP	openmrssdb:openmrs not reachable – nothing retired	Database still booting
GAP	N form(s) still live after the UPDATE	The write did not take

no-db is a GAP rather than a SKIP on purpose: the forms the release replaces are still being offered, and nothing else on the site will retire them.

The **nightly** `eregister-form-import` job does not retire. Retiring a form set is a release action, not something that should happen unattended at 03:30.

5.4 4.4 The import

5.4.1 Change detection

The importer keys its state file on **server URL + form name** and stores a sha256 of the source file:

```
{
  "https://localhost|HIV Treatment and Care Follow Up 2026": {
    "version": "3",
    "sha256": "ab12...",
    "form_uuid": "...",
    "file": "forms/HIV...json",
    "imported_at": "2026-08-24T09:12:03Z"
  }
}
```

So:

- a file **unchanged** since its last import → skipped, nothing deployed
- a same-named file holding a **new export** → counts as new work, deployed as the next version
- a file merely re-checked-out by the auto-pull job (new mtime, same bytes) → skipped

Hashing the export rather than the fixed-up payload means the question asked is "did the author change the form?", independent of concepts being re-resolved on every run.

Keying on the URL as well as the name keeps a dev run from convincing a later prod run that a form is already up to date.

5.4.2 Versioning

A changed form goes out as a **new version**; nothing live is overwritten.

last version 3 (state 3, server 3) – deploying 4

The target is $\max(\text{state version}, \text{server version}) + 1$. The state file is the record of what the script deployed, but the **server is the authority** — someone may have saved a newer version in the Implementer Interface, or the state file may have been lost.

5.4.3 Concept validation

A form whose concepts are not in the dictionary is **not** deployed. The run writes `<form>.importErrors.txt` into `/var/lib/v1/form-import/` listing the unresolved names. That is usually a sign the concept dictionary has not caught up — see [chapter 5](#).

`--skip-validation` bypasses it; `--dry-run` validates without deploying.

5.4.4 Concurrency

Two overlapping runs would both read the same "last version" and deploy the same number, so runs are serialised with flock on <state file>.lock. Where flock is absent (stock macOS) the lock is simply skipped.

5.4.5 Importer flags

```
/usr/local/bin/bahmni-form-import.sh [options] <file-or-dir>...
```

Flag	Meaning
--url URL	EMR base URL
--user / --password	Credentials
-k, --insecure	Accept the stack's self-signed certificate
-r, --recursive	Recurse into directories
-f, --force	Deploy even when the file is unchanged
--no-publish	Leave the form in Draft (what the Import button does)
--publish-only	Publish what is already deployed; import nothing
--dry-run	Validate concepts only; deploy nothing
--skip-validation	Deploy without resolving concept references
--version N	Pin the version instead of bumping
--no-bump	Do not increment the version
--state FILE	Use a different state file
--delay N	Pause between forms
-v, --verbose	More output

5.5 4.5 Publishing

This is the step the Implementer Interface leaves to you. Importing a form there puts it in **Draft**; you then click "Publish" separately, and until you do the form is not offered in the clinical app at all.

Deploying dozens of forms and then clicking Publish dozens of times is exactly the work this script exists to remove, so it publishes by default, over the same endpoint that button uses:

```
POST /openmrs/ws/rest/v1/bahmniie/form/publish?formUuid=<uuid>
```

Using the Bahmni endpoint rather than just flipping the published column means whatever else Bahmni does on publish comes along with it. Where that endpoint is missing — an older bahmnicore, or a stack without the module — the importer falls back to the core OpenMRS form resource:

```
POST /openmrs/ws/rest/v1/form/<uuid> {"published": true}
```

Either way the result is **read back** from the server before it is reported: a 200 from the publish endpoint is not the same thing as a published form.

```
form uuid 4f2a...
saved body, version 4
published version 4
imported 'ANC Intake 2026' as version 4
```

5.5.1 It re-asserts

A form that is skipped as **unchanged** is still checked, and published if it is not:

```
=== ANC Intake 2026.json
  unchanged since version 3 – skipping (--force to import anyway)
  published version 3
```

That is what fixes a site whose forms were deployed before any of this existed: they sit there as drafts, their files have not changed, so no ordinary import would ever touch them again. One normal catch-up run now publishes the lot.

It costs one GET per unchanged form, and a POST only when there is something to fix. A run where everything is already published makes no writes at all.

The flip side is deliberate. A form you unpublish by hand is published again by the next run. Use `--no-publish / EREGISTER_FORM_PUBLISH=0` on a site where that matters.

5.5.2 Publishing without importing

For a site whose forms are already deployed and only need publishing:

```
sudo /usr/local/bin/bahmni-form-import.sh -k --publish-only \  
-r /var/lib/v1/clinical-obs-forms
```

No concept resolution, no `POST /form`, no version bump — seconds per form rather than minutes, and it cannot change what any form contains.

```
=== ANC Intake 2026.json  
published version 3
```

```
=== HIV Follow Up 2026.json
```

```
published 1/2 form(s); nothing was imported (--publish-only)
```

`--dry-run` alongside it reports what each form's current state is without changing anything:

```
sudo /usr/local/bin/bahmni-form-import.sh -k --publish-only --dry-run \  
-r /var/lib/v1/clinical-obs-forms
```

5.5.3 Which version gets published

The **newest** version of that form name on the server. Older versions are left exactly as they are — publishing v4 does not touch v1–v3.

5.5.4 Turning it off

Where	How
catch-up.sh	<code>--no-publish</code> (and <code>--no-forms</code> implies it)
The importer	<code>--no-publish</code>
Anywhere	<code>EREGISTER_FORM_PUBLISH=0</code> , which reaches the daily job as <code>BAHMNI_PUBLISH=0</code> in <code>/etc/eregister</code>

5.6 4.6 Decoding the deployed form JSON

A deployed form can come back out of the EMR with the markup in its labels HTML-escaped — `<`; `>`; where the author wrote `<` `>`. The clinical app then renders the entity text itself, and forms that reference those fields throw errors.

The fix is textual: decode the three entities in place, in the folder the EMR keeps inside its own container (`/home/bahmni/clinical_forms` by default).

5.6.1 Why repeatedly

The escaping nests. `<`; `<`; is `<`; that was escaped a second time, and one pass over it only gets back as far as `<`; — so a single pass can leave a file that is still wrong, and still wrong in a way the same pass would fix.

It therefore runs again until a pass finds nothing left to change, rather than a fixed number of times: five levels of nesting take five passes, a site with one stops after the second, and `EREGISTER_FORM_DECODE_MAX_PASSES` (default 5) caps it so a pathological file cannot spin forever.

Idempotent, and cheap on a clean folder: the first pass matches nothing and it stops there. That is what makes it safe on every catch-up run.

It runs **before** the EMR is recreated at the end of a catch-up, so the reloaded instance reads the decoded files.

5.6.2 Return codes

Code	Meaning
0	Clean — nothing left escaped
1	Could not run (no docker compose, no stack dir, exec failed)
3	The folder does not exist inside the container — set EREGISTER_FORM_DECODE_DIR
4	Still escaped after the maximum passes — raise EREGISTER_FORM_DECODE_MAX_PASSES

5.6.3 Decode on its own

```
sudo ./catch-up.sh --decode
```

Seconds instead of a full run. See §3.6.

--decode and --no-decode are not a toggle pair. --no-decode skips this step inside a normal run; --decode makes this step the *only* thing the run does. Passing both is an error.

5.7 4.7 import-forms.sh — forms on their own

Sets up (or re-installs and re-schedules) the whole form-import machinery without the rest of the catch-up:

```
curl -fsSL --retry 8 --retry-max-time 180 \
```

```
↳ https://raw.githubusercontent.com/Lesotho-eRegister-v1/upgrade-to-v1/refs/heads/main/import-forms.sh
↳ | bash
```

or, from the checkout:

```
bash ./import-forms.sh
```

Note: this script does **not** retire forms. The retirement is a release action owned by catch-up.sh.

5.8 4.8 Running the import by hand

```
# the scheduled job, right now
```

```
sudo /usr/local/bin/eregister-form-import.sh
```

```
# validate concepts only, deploy nothing
```

```
sudo /usr/local/bin/bahmni-form-import.sh -k --dry-run -r /var/lib/v1/clinical-obs-forms
```

```
# re-deploy everything, ignoring the state file
```

```
sudo /usr/local/bin/bahmni-form-import.sh -k --force -r /var/lib/v1/clinical-obs-forms
```

```
# publish everything that is deployed but still in Draft
```

```
sudo /usr/local/bin/bahmni-form-import.sh -k --publish-only -r
```

```
↳ /var/lib/v1/clinical-obs-forms
```

```
# what happened last night
```

```
sudo tail -n 50 /var/log/eregister-form-import.log
```

```
# which forms does the site think it has deployed?
```

```
sudo jq 'keys' /var/lib/v1/.bahmni_form_import_state.json
```

5.8.1 Inspecting forms in the database

```
-- every version of the 2026 forms, live and retired, published or draft
```

```
SELECT form_id, name, version, published, retired, date_retired
FROM form WHERE name LIKE '%2026%' ORDER BY name, version;
```

```

-- anything still sitting in Draft
SELECT name, version FROM form
WHERE published = 0 AND retired = 0 ORDER BY name;

-- what this script retired
SELECT COUNT(*) FROM form
WHERE retire_reason = 'deploying latest forms with the latest changes - kgatman';

```

5.9 4.9 Configuration summary

Variable	Default
EREGISTER_IMPORT_FORMS	1 (--no-forms disables)
EREGISTER_BAHMNI_URL	https://localhost
EREGISTER_BAHMNI_USER	superman
EREGISTER_BAHMNI_PASS	(prompted)
EREGISTER_FORMS_DIR	<base>/v1/clinical-obs-forms
EREGISTER_FORM_IMPORT_INSECURE	1 (self-signed cert)
EREGISTER_FORM_IMPORT_CRON	30 3 * * *
EREGISTER_FORM_IMPORT_ONCALENDAR	*-*-* 03:30:00
EREGISTER_FORM_IMPORT_SELF_PULL	1 — refresh the clone before importing
EREGISTER_FORM_PUBLISH	1 (--no-publish disables)
EREGISTER_FORM_RETIRE	1 (--no-retire-forms disables)
EREGISTER_FORM_RETIRE_NAME_LIKE	%2026%
EREGISTER_FORM_RETIRE_REASON	deploying latest forms with the latest changes - kgatman
EREGISTER_FORM_RETIRE_BY	1
EREGISTER_FORM_DECODE	1 (--no-decode disables)
EREGISTER_FORM_DECODE_DIR	/home/bahmni/clinical_forms
EREGISTER_FORM_DECODE_MAX_PASSES	5

Chapter 6

5. The concept dictionary

The `eregister_concepts_release_v1` repo ships a **mysqldump of the OpenMRS concept dictionary** — `omrs_concept_dictionary_<timestamp>.sql`. It is loaded into the `openmrs` database inside the `openmrsdb` service.

Nothing imports this during the upgrade, and `catch-up.sh` only ever reports on it. The import drops and recreates whole tables, which is a big enough operation to belong to a job of its own, or to a deliberate manual run.

6.1 5.1 What the import actually does

The dump is a plain `mysqldump` with `FOREIGN_KEY_CHECKS=0`, `DROP TABLE IF EXISTS + CREATE TABLE + INSERTs`. Importing it therefore **replaces those tables wholesale** rather than merging into them.

The table list is not hard-coded. It is read straight out of the `.sql` file being imported:

```
sed -n 's/^DROP TABLE IF EXISTS `\[A-Za-z0-9_\]*\`.*$/p' <dump> | sort -u
```

so the pre-import backup always matches the file about to be imported, even when a new release adds a table.

drug_order is among those tables. Any drug orders the site currently holds are replaced. This is the single most important thing to understand about this import.

A pre-import dump of exactly those tables is written to `<base>/v1/bahmni-backup/concepts-preimport-<stamp>.sql` before anything is replaced, so the import can be undone by feeding that file back in.

6.2 5.2 Who imports it, and when

Path	When
Delayed first run	~3 hours after <code>install.sh</code> finishes, once, via a transient <code>systemd</code> timer (<code>eregister-concepts</code>)
The daily job	04:30 every night — but only when the dump's content differs from the one already loaded
<code>./import-concepts.sh</code>	Whenever you want, by hand
<code>catch-up.sh</code>	Never. It reports only

If neither `systemd` nor `at` is available, the daily job simply takes the first import.

6.2.1 Why the delay

When `install.sh` finishes, the stack has only just been started. `openmrsdb` answers early, but the instance behind it needs 30+ minutes — often hours on site hardware — before importing a dictionary is worth doing.

Doing it inline therefore either blocked the run or skipped for nothing, and either way it asked the operator to authorise replacing the `concept_*/drug*` tables at the least informative possible moment.

6.2.2 Change detection

The daily job records the sha256 of the dump it imported in `<base>/v1/.eregister_concept_import_state`. A night where the clone has not moved does nothing at all.

6.3 5.3 What catch-up.sh reports

```
✓ OK   concepts  dump      omrs_concept_dictionary_20260804.sql (newest of 3)
✓ OK   concepts  imported  sha256 matches the dump on disk
✓ OK   concepts  database  openmrs.concept holds 41,208 rows
```

The severity of the imported row depends on whether the daily job is actually scheduled:

- a **newer dump with the job in place** → OK. It is simply pending tonight.
- the **same dump with no job scheduled** → GAP. Nothing will ever load it.

`--no-concepts` silences these rows *and* leaves the daily job alone — reporting a gap that tells you to run the job you just disabled would be noise, and it would fail the exit code for a state you chose.

6.4 5.4 The OCL concept-name fix

Run **once**, after the v1 instance has fully started and finished its OCL import (~30+ minutes after the stack starts).

During that import OCL voids the local concept names (reason "Removed from OCL") and inserts its own replacements. `ocl-fix.sh`:

1. **Unvoids** the original names and **voids** OCL's replacements, in the openmrs database of the openmrssdb service.
2. **Renames** `CIEL_*.zip` → `.DONE` in the OCL directory of the openmrs service, so the import does not run again on the next startup.

Naturally idempotent: a second run finds nothing to unvoid and no zip to move.

```
./ocl-fix.sh [--yes] [--install-dir DIR] [--no-color] [--help]
```

```
curl -fsSL --retry 8 --retry-max-time 180 \
```

```
↪ https://raw.githubusercontent.com/Lesotho-eRegister-v1/upgrade-to-v1/refs/heads/main/ocl-fix.sh
↪ | bash
```

Variable	Default
EREGISTER_INSTALL_BASE	/var/lib
EREGISTER_DB_SERVICE	openmrssdb
EREGISTER_EMR_SERVICE	openmrs
EREGISTER_OCL_DIR	/openmrs/data/configuration/ocl
EREGISTER_DB_PASS	(the container's own MYSQL_ROOT_PASSWORD)

6.5 5.5 import-concepts.sh

```
./import-concepts.sh [--yes] [--install-dir DIR] [--no-color] [--help]
./import-concepts.sh --schedule # install the DAILY job instead of importing now
```

Import now:

```
sudo ./import-concepts.sh
```

Install (or re-install) the daily job without importing:

```
sudo ./import-concepts.sh --schedule
```

From the network:


```
curl -fsSL --retry 8 --retry-max-time 180 \
↳ https://raw.githubusercontent.com/Lesotho-eRegister-v1/upgrade-to-v1/refs/heads/main/import-concepts.
↳ | bash
```

6.5.1 Before it writes, it warns

```
[Δ] This REPLACES the concept dictionary in the 'openmrs' database with
[Δ] the dump in
↳ /var/lib/v1/eregister_concepts_release_v1/omrs_concept_dictionary_20260804.sql.
[Δ] The tables it drops and recreates include drug_order – so any rows those
[Δ] tables currently hold are replaced.
[i] Backing up the tables about to be replaced ->
↳ /var/lib/v1/bahmni-backup/concepts-preimport-20260921_141203.sql
```

6.5.2 Undoing an import

```
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard
sudo cat /var/lib/v1/bahmni-backup/concepts-preimport-<stamp>.sql \
| sudo docker compose exec -T openmrssdb mysql -uroot -p openmrs
```

6.6 5.6 Choosing which dump to import

The repo's own tooling names each release with a timestamp (run_concept_dump.sh writes omrs_concept_dictionary_\$(date +%Y%m%d_%H%M%S).sql), so there is no fixed filename to pin.

- Leave EREGISTER_CONCEPTS_SQL_NAME **empty** (the normal case) to pick the newest file matching EREGISTER_CONCEPTS_SQL_PATTERN (default omrs_concept_dictionary*.sql).
- Set it to pin one exact filename.

6.7 5.7 Database connectivity

Every step that talks to the database asks **once** and does not poll:

```
[Δ] openmrssdb:openmrs is not accepting connections right now.
[Δ] A freshly started v1 stack needs 30+ minutes before its database answers,
[Δ] so this is expected straight after an upgrade – it is not an error.
[Δ] Skipping the concept dictionary; nothing else in this run depends on it.
[Δ] Load it once the stack is up with: ./import-concepts.sh
```

This used to poll for up to five minutes, on the theory that the import runs while the stack is still booting. In practice that turned a skippable step into a five-minute hang whenever openmrssdb was not coming up at all – and the import is not on the critical path. So: ask once, and if the answer is no, say why and move on.

The password comes from EREGISTER_DB_PASS when set, else the container's own MYSQL_ROOT_PASSWORD, so it never lands on a host process list.

6.8 5.8 Configuration summary

Variable	Default	Meaning
EREGISTER_IMPORT_CONCEPTS	1	0 (or --no-concepts) disables
EREGISTER_CONCEPTS_SQL_NAME	(empty)	Pin one exact dump filename
EREGISTER_CONCEPTS_SQL_PATTERN	omrs_concept_dictionary*.sql	Which files to consider
EREGISTER_CONCEPT_IMPORT	1	0 does not install the daily j
EREGISTER_CONCEPT_IMPORT_CRON	30 4 * * *	cron schedule
EREGISTER_CONCEPT_IMPORT_ONCALENDAR	*-*-* 04:30:00	systemd schedule
EREGISTER_CONCEPT_IMPORT_FIRST_RUN	1	0 = no delayed first run
EREGISTER_CONCEPT_IMPORT_FIRST_DELAY_SEC	10800 (3h)	How long to wait for it
EREGISTER_CONCEPT_IMPORT_RESTART_EMR	0	Restart the EMR after an im

An imported dictionary is only visible after the EMR restarts.

Chapter 7

6. OpenMRS report definitions

The `openmrs_reporting_release` repo ships a **mysqldump of the OpenMRS reporting module's `serialized_object` table** — currently `Serialized_Object.sql`.

That table *is* the report library: every report, cohort, indicator and dataset definition the Reports app offers is one serialized XML row in it.

7.1 6.1 The one import catch-up performs itself

`catch-up.sh` imports this directly, where it only ever **reports** on the concept dictionary. It earns that difference:

	Concept dictionary	Report definitions
Tables replaced	<code>concept_*</code> , <code>drug*</code> , including <code>drug_order</code>	<code>serialized_object</code> only
Patient data touched	Drug orders are replaced	None
Blast radius	Large	One table of report definitions
Undo	Pre-import dump	Pre-import dump
Who imports it	The daily job, or you	<code>catch-up.sh</code>

Importing **replaces** the table rather than merging into it. Report definitions written on this site by hand, and not present in the release, are lost. The pre-import dump is what gets them back.

7.2 6.2 What one run does

1. Resolve which `.sql` file(s) to import
2. Hash them and compare against `<base>/v1/.eregister_reporting_import_state`
3. **If unchanged — stop.** Nothing is touched
4. Dump the current `serialized_object` to `<base>/v1/bahmni-backup/reporting-preimport-<stamp>.sql`
5. Feed the release dump into `openmrsdb:openmrs`
6. Record the new hash

Re-runnable by design: a second run with an unchanged clone does nothing at all, and a run after the auto-pull job brings in a new release imports the new definitions.

7.3 6.3 Which files get imported

- Leave `EREGISTER_REPORTING_SQL_NAME` **empty** (the normal case) to import every file matching `EREGISTER_REPORTING_SQL_PATTERN` (default `*.sql`) at the top of the clone, in filename order. A second dump added upstream is then picked up without a code change.
- Set it to pin one exact filename.

7.4 6.4 Report rows

```
✓ OK      reporting import    already current (sha256 matches the clone)
○ FIXED   reporting import    imported Serialized_Object.sql – pre-import dump in
  ↳ /var/lib/v1/bahmni-backup/
✗ GAP     reporting import    openmrddb:openmrs not reachable – re-run catch-up once the
  ↳ stack is up
– SKIP    reporting import    disabled (--no-reporting)
```

--no-reporting still clones and fast-forwards the repo; only the database import is skipped.

7.5 6.5 Forcing and undoing

```
# re-import (skipped when already current)
sudo /var/lib/v1/upgrade-to-v1/catch-up.sh

# force one: drop the state marker, then run again
sudo rm /var/lib/v1/.eregister_reporting_import_state
sudo /var/lib/v1/upgrade-to-v1/catch-up.sh

# undo one
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard
sudo cat /var/lib/v1/bahmni-backup/reporting-preimport-<stamp>.sql \
| sudo docker compose exec -T openmrddb mysql -uroot -p openmrs
```

New reports only appear after the EMR restarts. The catch-up run's final step does that for you unless --no-recreate was passed.

7.6 6.6 Configuration summary

Variable	Default	Meaning
EREGISTER_IMPORT_REPORTING	1	0 (or --no-reporting) skips the import
EREGISTER_REPORTING_SQL_NAME	(empty)	Pin one exact filename
EREGISTER_REPORTING_SQL_PATTERN	*.sql	Which files to consider
EREGISTER_REF_REPORTING	master	The repo's branch

Chapter 8

7. Database backups

There are **two** different backups in this toolkit, and confusing them is the easiest way to lose data.

	The pre-upgrade dump	The rolling nightly backup
Taken	Once , by <code>install.sh</code> , from the 0.92 stack	Every night at 01:30, from the live v1 stack
Lives in	<code><base>/v1/bahmni-backup/openmrstdb_backup.sql</code>	<code><base>/v1/db-backups/openmrs_<stamp>.s</code>
Read by	<code>restore_bahmni_standard.sh</code> during the upgrade	You, by hand
Retention	Kept forever	Newest 14
Purpose	The 0.92 data the upgrade migrates	Undo a bad import or a deleted record

`install.sh --force` reloads the database from the pre-upgrade dump, replacing anything entered since it was made. On a site that has been live on v1 for a while, that is data loss.

8.1 7.1 The nightly backup

Installed by `install.sh` (first, before anything else that writes to the database on a schedule) and checked by every `catch-up.sh` run.

```
sudo /usr/local/bin/eregister-db-backup.sh    # run it now
```

8.1.1 What one run does

1. `mysqldump` the whole `openmrs` database inside the `openmrstdb` container, with `--single-transaction` — a consistent snapshot without locking writers
2. Stream it out, `gzip` it host-side, write it under a `.part` name
3. Check **both** the `gzip` integrity **and** that `mysqldump` wrote its `Dump completed trailer`
4. Only then move it into place and repoint `latest.sql.gz`
5. Delete all but the newest `EREGISTER_DB_BACKUP_KEEP` dumps

Step 3 is the one that matters. A dump cut short by an OOM or a restarting container is otherwise a perfectly plausible-looking file that restores half a site — and looks exactly like a good one until the day you need it.

8.1.2 Why 01:30

Before the auto-pull (02:30), the form import (03:30) and the concept import (04:30), so each night's dump is of the database as it was **before** anything scheduled touched it.

8.1.3 The script is standalone on purpose

`cron` and `systemd` run it in a bare environment, so it carries its own logger, resolves `docker` `compose` itself, and depends on nothing under `lib/`.

Mode 0755 normally. When EREGISTER_DB_PASS was set, the password is baked into the file and it goes in 0700 instead — the usual case bakes nothing, because the v1 database's root password is the container's own MYSQL_ROOT_PASSWORD and the script reads it there.

8.2 7.2 What catch-up checks

Two separate questions, two separate rows:

```
✓ OK   cron   eregister-db-backup   systemd timer active (next: Mon 01:30)
✗ GAP  backup newest dump           newest dump is 51h old — the nightly job is not
    ↪ producing files
```

The first says a timer **exists**. The second says the timer is **producing files**, which is the question that matters: a backup job whose openmrsdb service has been unreachable since a compose rename fails silently every night, and the only visible symptom is that the newest dump stops moving.

A newest dump over **36h** old is a GAP. So is any leftover .part file — that is a run that was interrupted mid-write.

--no-db-backup silences both rows and leaves the job alone.

8.3 7.3 Restoring

```
gzip -dc /var/lib/v1/db-backups/latest.sql.gz \
| ( cd /var/lib/v1/bahmni-docker-ls/bahmni-standard \
  && sudo docker compose exec -T openmrsdb mysql -uroot -p )
```

The dump carries CREATE DATABASE / USE, so no database name is needed on the command line.

8.3.1 Verifying a dump before you trust it

```
# integrity of the gzip stream
gzip -t /var/lib/v1/db-backups/openmrs_20260921_013001.sql.gz && echo "gzip OK"

# did mysqldump actually finish?
gzip -dc /var/lib/v1/db-backups/openmrs_20260921_013001.sql.gz | tail -n 3
# -> should end with: -- Dump completed on 2026-09-21 1:34:12
```

8.4 7.4 These dumps do not protect you from losing the machine

They sit on the **same disk as the database**. They protect against a bad import or a deleted record, not against a failed disk or a lost host.

Copy <base>/v1/db-backups off the host as well. Nothing in this toolkit does that for you.

8.5 7.5 Configuration summary

Variable	Default	Meaning
EREGISTER_DB_BACKUP	1	0 (or --no-db-backup) does not in
EREGISTER_DB_BACKUP_CRON	30 1 * * *	cron schedule
EREGISTER_DB_BACKUP_ONCALENDAR	*-*-* 01:30:00	systemd schedule
EREGISTER_DB_BACKUP_KEEP	14	How many dumps to retain
EREGISTER_DB_BACKUP_DIR	<base>/v1/db-backups	Where they go (mode 0700)
EREGISTER_DB_BACKUP_LOG	/var/log/eregister-db-backup.log	The log

Chapter 9

8. Scheduled jobs

Four jobs keep a v1 site current. They are independent of each other, and each can be run by hand at any time.

Time	Job	Unit / runner
01:30	Database backup	eregister-db-backup /usr/local/bin/eregister-db-backup.sh
02:30	Repo auto-pull	eregister-autopull /usr/local/bin/eregister-autopull.sh
03:30	Clinical form import	eregister-form-import /usr/local/bin/eregister-form-import.sh
04:30	Concept dictionary import	eregister-concept-import /usr/local/bin/eregister-concept-import.sh

The order is deliberate:

- The **backup** runs first, so the night's dump predates whatever the other three do.
- The **auto-pull** runs before the two imports, so each import sees the repos that were pulled the same night.

9.1 8.1 systemd or cron

Where systemd is present (Ubuntu's default), each job is a `.service + .timer` pair in `/etc/systemd/system/`. Where it is not, the job is an `/etc/cron.d/` entry.

Both run the **same** standalone runner script, so behaviour does not change with the scheduler.

A host with neither gets the exact cron line to add by hand:

```
[Δ] Neither systemd nor /etc/cron.d is available. The runner was installed at
[Δ] /usr/local/bin/eregister-form-import.sh but NOT scheduled – add your own entry:
[Δ] 30 3 * * * root /usr/local/bin/eregister-form-import.sh
```

9.2 8.2 The auto-pull job

Keeps the asset and config repos in sync with their remotes — `fetch --depth 1` plus a fast-forward reset —hard onto the tracked branch, which mirrors what the installer does and keeps the checkout shallow.

Covered repos:

```
standard-config-ls
implementer-interface-release
openmrs-v1-modules
clinical-obs-forms
dhisconnector_mappings_v1
eregister_concepts_release_v1
openmrs_reporting_release
```

Deliberately excluded:

- `bahmni-docker-ls` — the stack itself, pinned to the deployed release. It must not drift underneath a running instance.
- `bahmni_config` (0.92) — historical, restore-only.

A **dirty working tree is left untouched**. Never clobber uncommitted local work.

Pulling a repo changes files on disk. It does **not** deploy anything. The forms still need an import, the dictionary still needs a load, and the stack still needs `docker compose up -d`. That is what the nightly imports and `catch-up.sh` are for.

9.3 8.3 What catch-up checks

For each job, one row:

```
✓ OK    cron  eregister-db-backup      systemd timer active (next: Mon 2026-09-22
↳ 01:30:00)
○ FIXED cron  eregister-form-import    was missing — installed (systemd timer)
- SKIP  cron  eregister-concept-import disabled (--no-concepts)
✗ GAP   cron  eregister-autopull     not scheduled (declined)
```

Note the shape of the form-importer rows in particular: neither `--no-forms` nor a failed importer install may skip the *credentials* row. A site that asked to leave its forms alone still needs the script and env file in place for a later manual run.

9.4 8.4 Inspecting a job

9.4.1 systemd

is the timer active, and when does it next fire?

```
systemctl status eregister-form-import.timer
systemctl list-timers 'eregister-*
```

what happened on the last run?

```
journalctl -u eregister-form-import.service -n 50
```

run it now, out of schedule

```
sudo systemctl start eregister-form-import.service
```

9.4.2 cron

```
cat /etc/cron.d/eregister-form-import
sudo tail -n 50 /var/log/eregister-form-import.log
```

9.4.3 Either way

Every runner is safe to invoke directly:

```
sudo /usr/local/bin/eregister-db-backup.sh
sudo /usr/local/bin/eregister-autopull.sh
sudo /usr/local/bin/eregister-form-import.sh
sudo /usr/local/bin/eregister-concept-import.sh
```

9.5 8.5 Changing a schedule

Set the environment variable and re-run `catch-up.sh`, which rewrites the unit:

```
# form import at 05:15 instead of 03:30
sudo EREGISTER_FORM_IMPORT_ONCALENDAR='*-*-* 05:15:00' \
    EREGISTER_FORM_IMPORT_CRON='15 5 * * *' \
    ./catch-up.sh
```

Set **both** variables. The `systemd` one is used where `systemd` is present, the `cron` one where it is not, and a host can change scheduler between runs.

9.6 8.6 Disabling a job

Job	Disable with
Database backup	--no-db-backup / EREGISTER_DB_BACKUP=0
Auto-pull	EREGISTER_AUTO_PULL=0
Form import	--no-forms / EREGISTER_IMPORT_FORMS=0
Concept import	--no-concepts / EREGISTER_CONCEPT_IMPORT=0

Disabling silences the corresponding report rows too, so the exit code does not fail for a state you chose.

9.7 8.7 The form-import runner in detail

The wrapper `eregister-form-import.sh` is generated, and re-generated by every `catch-up.sh` run. One run:

1. Sources `/etc/eregister/form-import.env` for the credentials
2. If `EREGISTER_FORM_IMPORT_SELF_PULL=1` (the default), refreshes the `clinical-obs-forms` clone itself — so the daily import still picks up new forms on a host where auto-pull was declined
3. cds into `/var/lib/v1/form-import` so `importErrors.txt` lands somewhere writable rather than wherever cron happened to start
4. Runs the importer over the whole folder with `-k -r`

A dirty tree or a detached HEAD skips the refresh and imports what is on disk:

```
SKIP refresh (/var/lib/v1/clinical-obs-forms has uncommitted local changes)
```

Chapter 10

9. Configuration reference

Everything is configured in one of two ways: a **command-line flag**, or an EREGISTER_* **environment variable**. Where both exist, the flag is a shorthand for the variable and the two are listed together.

Variables are read at startup, so they work equally well inline:

```
sudo EREGISTER_FORM_RETIRE_NAME_LIKE='%2027%' ./catch-up.sh
```

...exported:

```
export EREGISTER_INSTALL_BASE=/srv
```

```
sudo -E ./catch-up.sh
```

...or through a piped one-liner:

```
curl -fsSL <raw>/catch-up.sh | sudo EREGISTER_BAHMNI_PASS='...' bash -s -- --yes
```

10.1 9.1 Flags by script

10.1.1 install.sh

Flag	Variable	Default
-y, --yes	EREGISTER_ASSUME_YES=1	0
--force	—	off
--install-dir DIR	EREGISTER_INSTALL_BASE	/var/lib
--target-ref REF[,...]	EREGISTER_TARGET_REF	(empty — per-repo defaults)
--no-concepts	EREGISTER_IMPORT_CONCEPTS=0	1
--no-db-backup	EREGISTER_DB_BACKUP=0	1
--no-forms	—	<i>inert in this script</i>
--no-color	—	auto
-h, --help	—	—

10.1.2 catch-up.sh

Flag	Variable	Default
-y, --yes	EREGISTER_ASSUME_YES=1	0
--decode, --decode-only	EREGISTER_CATCHUP_DECODE_ONLY=1	0
--no-stack	EREGISTER_CATCHUP_STACK_REPO=0	1
--no-forms	EREGISTER_IMPORT_FORMS=0	1
--no-retire-forms	EREGISTER_FORM_RETIRE=0	1
--no-publish	EREGISTER_FORM_PUBLISH=0	1
--no-decode	EREGISTER_FORM_DECODE=0	1
--no-concepts	EREGISTER_CATCHUP_DB_CHECK=0 + EREGISTER_CONCEPT_IMPORT=0	1
--no-reporting	EREGISTER_IMPORT_REPORTING=0	1

Flag	Variable	Default
--no-idgen	EREGISTER_IDGEN_RETIRE=0	1
--no-db-backup	EREGISTER_DB_BACKUP=0	1
--no-compose-up	EREGISTER_CATCHUP_COMPOSE_UP=0	1
--pull-images	EREGISTER_CATCHUP_COMPOSE_PULL=1	0
--no-recreate	EREGISTER_CATCHUP_RECREATE=0	1
--force-repos	EREGISTER_CATCHUP_FORCE_REPOS=1	0
--install-dir DIR	EREGISTER_INSTALL_BASE	/var/lib
--no-color	—	auto

--no-forms implies --no-decode, --no-retire-forms **and** --no-publish. --decode and --no-decode together are an error.

10.1.3 import-concepts.sh

Flag	Meaning
--schedule	Install the daily job instead of importing now
--yes, --install-dir DIR, --no-color, --help	As above

10.1.4 import-forms.sh, ocl-fix.sh

--yes, --install-dir DIR, --no-color, --help.

10.2 9.2 Core

Variable	Default	Meaning
EREGISTER_INSTALL_BASE	/var/lib	Install base; everything lives in <base>/v1
EREGISTER_ASSUME_YES	0	Non-interactive
EREGISTER_TARGET_REF	(empty)	Comma-separated ref preference list for every repo
EREGISTER_LIB_DIR	(beside the script)	Where to find lib/
EREGISTER_RAW_BASE	the repo's raw main URL	Where to fetch modules from

10.3 9.3 The 0.92 source stack

Variable	Default	Meaning
EREGISTER_OLD_DOCKER_DIR	\$HOME/bahmni_docker	Where the 0.92 stack lives
EREGISTER_EMR_CONTAINER	bahmni_docker-emr-service-1	The 0.92 EMR container
EREGISTER_EMR_CONTAINER_ALT	bahmni_docker_emr_service_1	Fallback name
EREGISTER_DB_NAME	openmrs	Database to dump and restore
EREGISTER_DB_USER	root	Database user
EREGISTER_DB_PASS	(prompted)	Database password

10.4 9.4 The v1 stack

Variable	Default	Meaning
EREGISTER_DB_SERVICE	openmrsdb	Compose service hosting the database
EREGISTER_EMR_SERVICE	openmrs	Compose service hosting the EMR
EREGISTER_REPORTS_SERVICE	reports	Compose service for reports
EREGISTER_OCL_DIR	/openmrs/data/configuration/ocl	OCL dir <i>inside</i> the EMR service

10.5 9.5 Repository refs

Variable	Default
EREGISTER_REF_BAHMNI_DOCKER	Bokang-changes
EREGISTER_REF_STANDARD_CONFIG	Bokang-changes
EREGISTER_REF_CONFIG_092	main
EREGISTER_REF_OPENMRS_MODULES	main
EREGISTER_REF_IMPL_INTERFACE	main
EREGISTER_REF_OBS_FORMS	main
EREGISTER_REF_DHIS_MAPPINGS	master
EREGISTER_REF_CONCEPTS	main
EREGISTER_REF_REPORTING	master
EREGISTER_UPGRADE_REPO	https://github.com/Lesotho-eRegister-v1/upgrade-to-v1
EREGISTER_UPGRADE_REF	main
EREGISTER_UPGRADE_REPO_DIR	<base>/v1/upgrade-to-v1

EREGISTER_TARGET_REF supersedes all of the per-repo refs.

10.6 9.6 Clinical forms

See [chapter 4](#) for what each one does.

Variable	Default
EREGISTER_IMPORT_FORMS	1
EREGISTER_BAHMNI_URL	https://localhost
EREGISTER_BAHMNI_USER	superman
EREGISTER_BAHMNI_PASS	(prompted)
EREGISTER_FORMS_DIR	<base>/v1/clinical-obs-forms
EREGISTER_FORM_IMPORT_INSECURE	1
EREGISTER_FORM_IMPORT_SCRIPT	/usr/local/bin/bahmni-form-import.sh
EREGISTER_FORM_IMPORT_RUNNER	/usr/local/bin/eregister-form-import.sh
EREGISTER_FORM_IMPORT_ENV	/etc/eregister/form-import.env
EREGISTER_FORM_IMPORT_LOG	/var/log/eregister-form-import.log
EREGISTER_FORM_IMPORT_UNIT	eregister-form-import
EREGISTER_FORM_IMPORT_CRON	30 3 * * *
EREGISTER_FORM_IMPORT_ONCALENDAR	*--* 03:30:00
EREGISTER_FORM_IMPORT_SELF_PULL	1
EREGISTER_FORM_PUBLISH	1
EREGISTER_FORM_RETIRE	1
EREGISTER_FORM_RETIRE_NAME_LIKE	%2026%
EREGISTER_FORM_RETIRE_REASON	deploying latest forms with the latest changes – kgatman
EREGISTER_FORM_RETIRE_BY	1
EREGISTER_FORM_DECODE	1
EREGISTER_FORM_DECODE_DIR	/home/bahmni/clinical_forms
EREGISTER_FORM_DECODE_MAX_PASSES	5

10.7 9.7 Concept dictionary

Variable	Default
EREGISTER_IMPORT_CONCEPTS	1
EREGISTER_CONCEPTS_SQL_NAME	(empty — newest match wins)
EREGISTER_CONCEPTS_SQL_PATTERN	omrs_concept_dictionary*.sql
EREGISTER_CONCEPT_IMPORT	1

Variable	Default
EREGISTER_CONCEPT_IMPORT_RUNNER	/usr/local/bin/eregister-concept-import.sh
EREGISTER_CONCEPT_IMPORT_LOG	/var/log/eregister-concept-import.log
EREGISTER_CONCEPT_IMPORT_UNIT	eregister-concept-import
EREGISTER_CONCEPT_IMPORT_CRON	30 4 * * *
EREGISTER_CONCEPT_IMPORT_ONCALENDAR	*-*-* 04:30:00
EREGISTER_CONCEPT_IMPORT_SELF_PULL	1
EREGISTER_CONCEPT_IMPORT_FIRST_RUN	1
EREGISTER_CONCEPT_IMPORT_FIRST_DELAY_SEC	10800
EREGISTER_CONCEPT_IMPORT_RESTART_EMR	0

10.8 9.8 Report definitions

Variable	Default
EREGISTER_IMPORT_REPORTING	1
EREGISTER_REPORTING_SQL_NAME	(empty — every match)
EREGISTER_REPORTING_SQL_PATTERN	*.sql

10.9 9.9 Identifier source

Variable	Default
EREGISTER_IDGEN_RETIRE	1
EREGISTER_IDGEN_RETIRE_ID	14
EREGISTER_IDGEN_RETIRE_REASON	No longer in use
EREGISTER_IDGEN_RETIRE_BY	1

10.10 9.10 Database backups

Variable	Default
EREGISTER_DB_BACKUP	1
EREGISTER_DB_BACKUP_DIR	<base>/v1/db-backups
EREGISTER_DB_BACKUP_RUNNER	/usr/local/bin/eregister-db-backup.sh
EREGISTER_DB_BACKUP_LOG	/var/log/eregister-db-backup.log
EREGISTER_DB_BACKUP_UNIT	eregister-db-backup
EREGISTER_DB_BACKUP_CRON	30 1 * * *
EREGISTER_DB_BACKUP_ONCALENDAR	*-*-* 01:30:00
EREGISTER_DB_BACKUP_KEEP	14
EREGISTER_DB_BACKUP_COMPRESS	1
EREGISTER_DB_BACKUP_FIRST_RUN	1

10.11 9.11 Auto-pull

Variable	Default
EREGISTER_AUTO_PULL	1
EREGISTER_AUTO_PULL_SCRIPT	/usr/local/bin/eregister-autopull.sh
EREGISTER_AUTO_PULL_LOG	/var/log/eregister-autopull.log
EREGISTER_AUTO_PULL_UNIT	eregister-autopull
EREGISTER_AUTO_PULL_CRON	30 2 * * *
EREGISTER_AUTO_PULL_ONCALENDAR	*-*-* 02:30:00

Variable	Default
----------	---------

10.12 9.12 Catch-up behaviour

Variable	Default
EREGISTER_CATCHUP_STACK_REPO	1
EREGISTER_CATCHUP_DB_CHECK	1
EREGISTER_CATCHUP_DECODE_ONLY	0
EREGISTER_CATCHUP_COMPOSE_UP	1
EREGISTER_CATCHUP_COMPOSE_PULL	0
EREGISTER_CATCHUP_RECREATE	1
EREGISTER_CATCHUP_FORCE_REPOS	0
EREGISTER_CATCHUP_HTTP_TIMEOUT	15

Chapter 11

10. Operations runbook

Worked examples for the tasks that actually come up. Each one is a complete recipe, not a fragment.

11.1 10.1 Upgrade a 0.92 site to v1

1. From the network, interactively – you confirm every step

```
curl -fsSL --retry 8 --retry-max-time 180 \
```

```
↳ https://raw.githubusercontent.com/Lesotho-eRegister-v1/upgrade-to-v1/refs/heads/main/install.sh
↳ | bash
```

Then, **after waiting 30+ minutes for the EMR to boot:**

2. Fix the OCL concept names

```
cd /var/lib/v1/upgrade-to-v1 && sudo ./ocl-fix.sh
```

3. Deploy the forms and the report definitions

```
sudo ./catch-up.sh
```

Step 3 is not optional. `install.sh` clones the forms and the report definitions but imports neither. Until `catch-up.sh` has run, neither is on the site.

11.2 10.2 Deploy a new release of the 2026 forms

The normal case. Someone has pushed new form exports to `clinical-obs-forms`.

```
cd /var/lib/v1/upgrade-to-v1
```

```
sudo ./catch-up.sh
```

Answer yes at:

- Retire these N form(s) before importing? — retires the outgoing set
- Run the form import now? — deploys the incoming set
- Recreate 'openmrs' now? — reloads the EMR so it serves them

If you only want the forms touched and nothing else:

```
sudo ./catch-up.sh --no-stack --no-concepts --no-reporting \
                  --no-idgen --no-db-backup --no-compose-up
```

11.2.1 Verify afterwards

```
sudo tail -n 20 /var/log/eregister-form-import.log
```

```
SELECT name, version, retired FROM form
WHERE name LIKE '%2026%' ORDER BY name, version DESC;
```

11.3 10.3 Change the form-retirement pattern for a new year

```
sudo EREGISTER_FORM_RETIRE_NAME_LIKE='%2027%' \  
    EREGISTER_FORM_RETIRE_REASON='deploying 2027 forms' \  
    ./catch-up.sh
```

To make it permanent, change the default in `lib/core/config.sh` and push.

11.4 10.4 Forms are deployed but show as unpublished

A form imported through the Implementer Interface — or by a version of this script from before publishing existed — sits in **Draft**, and is not offered in the clinical app until it is published.

Publish everything that is deployed, without re-importing anything:

```
sudo /usr/local/bin/bahmni-form-import.sh -k --publish-only \  
    -r /var/lib/v1/clinical-obs-forms
```

Seconds per form, and it cannot change what any form contains. To see what would happen first:

```
sudo /usr/local/bin/bahmni-form-import.sh -k --publish-only --dry-run \  
    -r /var/lib/v1/clinical-obs-forms
```

An ordinary `catch-up.sh` run fixes this too — publication is re-asserted on forms that are skipped as unchanged — but `--publish-only` is the targeted, fast version.

11.4.1 Check what is still in Draft

```
SELECT name, version FROM form  
WHERE published = 0 AND retired = 0 ORDER BY name;
```

11.4.2 Keep a particular form unpublished

Publication re-asserts, so a hand-unpublished form comes back. On a site where that matters:

```
sudo ./catch-up.sh --no-publish
```

11.5 10.5 A form did not deploy

```
# 1. What did the importer say?  
sudo tail -n 60 /var/log/eregister-form-import.log
```

```
# 2. Unresolved concepts?  
ls -l /var/lib/v1/form-import/*.importErrors.txt  
cat /var/lib/v1/form-import/*.importErrors.txt
```

If concepts are missing, the dictionary has not caught up:

```
sudo ./import-concepts.sh          # then re-run the form import
```

If the form is simply being skipped as unchanged:

```
sudo /usr/local/bin/bahmni-form-import.sh -k --force \  
    -r /var/lib/v1/clinical-obs-forms
```

11.6 10.6 Forms render <t; instead of <

```
sudo ./catch-up.sh --decode
```

Seconds, not minutes. Then restart the EMR so it re-reads the files:

```
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard  
sudo docker compose restart openmrs
```

If a second decode run still reports escaping, the nesting is deeper than the default five passes:


```
sudo EREGISTER_FORM_DECODE_MAX_PASSES=12 ./catch-up.sh --decode
```

11.7 10.7 Pick up a new stack release

Someone has pushed new compose files to bahmni-docker-ls.

```
# in a maintenance window
cd /var/lib/v1/upgrade-to-v1
sudo ./catch-up.sh --pull-images
```

--pull-images matters when the release moves an image tag *in place*. Without it, the host deploys images it already has.

To check what moved before applying anything:

```
sudo ./catch-up.sh --no-compose-up --no-recreate
# read the 'repo bahmni-docker-ls' row, then decide
```

11.8 10.8 Undo a form retirement

```
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard
sudo docker compose exec -T openmrsdb mysql -uroot -p openmrs -e "
    UPDATE form SET retired = 0, retired_by = NULL, date_retired = NULL,
        retire_reason = NULL
    WHERE retire_reason = 'deploying latest forms with the latest changes - kgatman';"
```

Then pass --no-retire-forms on subsequent runs, or the next catch-up retires them again.

11.9 10.9 Undo an identifier-source retirement

```
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard
sudo docker compose exec -T openmrsdb mysql -uroot -p openmrs -e "
    UPDATE idgen_identifier_source
    SET retired = 0, retired_by = NULL, date_retired = NULL, retire_reason = NULL
    WHERE id = 14;"
```

Then pass --no-idgen, for the same reason.

To see the table first:

```
SELECT id, name, retired FROM idgen_identifier_source;
```

11.10 10.10 Undo a report-definition import

```
ls -lt /var/lib/v1/bahmni-backup/reporting-preimport-*.sql | head

cd /var/lib/v1/bahmni-docker-ls/bahmni-standard
sudo cat /var/lib/v1/bahmni-backup/reporting-preimport-20260921_141203.sql \
| sudo docker compose exec -T openmrsdb mysql -uroot -p openmrs
```

11.11 10.11 Undo a concept-dictionary import

```
ls -lt /var/lib/v1/bahmni-backup/concepts-preimport-*.sql | head

cd /var/lib/v1/bahmni-docker-ls/bahmni-standard
sudo cat /var/lib/v1/bahmni-backup/concepts-preimport-20260921_040501.sql \
| sudo docker compose exec -T openmrsdb mysql -uroot -p openmrs
```

11.12 10.12 Restore the database from a nightly dump

```
# check it first
gzip -t /var/lib/v1/db-backups/latest.sql.gz && echo "gzip OK"
gzip -dc /var/lib/v1/db-backups/latest.sql.gz | tail -n 3 # -> "Dump completed"

# restore
gzip -dc /var/lib/v1/db-backups/latest.sql.gz \
| ( cd /var/lib/v1/bahmni-docker-ls/bahmni-standard \
    && sudo docker compose exec -T openmrstdb mysql -uroot -p )

# then reload the EMR
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard
sudo docker compose up -d --force-recreate --renew-anon-volumes openmrs
```

11.13 10.13 Wire catch-up into monitoring

```
sudo crontab -e

30 6 * * 1 /var/lib/v1/upgrade-to-v1/catch-up.sh --yes --no-forms --no-idgen \
--no-compose-up --no-recreate >> /var/log/eregister-catchup.log 2>&1
```

Exit status is 0 only when there are no GAP rows, so any monitoring system that watches exit codes works unchanged.

Keep every one of those skips. --yes answers every confirmation, so without them a scheduled run would retire and re-import forms, retire the identifier source, apply the compose files and reload the EMR unattended — at 06:30 on a Monday, with nobody watching.

11.14 10.14 Re-point a site at a different branch

```
sudo EREGISTER_REF_OBS_FORMS=testing ./catch-up.sh
```

The repo is currently on main, so catch-up reports it as off-release and leaves it alone. Force the move:

```
sudo EREGISTER_REF_OBS_FORMS=testing ./catch-up.sh --force-repos
```

To move **every** repo at once:

```
sudo ./catch-up.sh --force-repos --install-dir /var/lib
# with EREGISTER_TARGET_REF set, if you want a single ref list
```

11.15 10.15 Move the whole install somewhere else

```
sudo ./install.sh --install-dir /srv
# everything then lives under /srv/v1
```

Every later run needs the same flag:

```
sudo ./catch-up.sh --install-dir /srv
```

or set it once:

```
export EREGISTER_INSTALL_BASE=/srv
```

11.16 10.16 Re-run an interrupted upgrade

```
cat /var/lib/v1/.eregister-upgrade-complete
```

Contents	Do this
stage=complete	Nothing. Run catch-up.sh instead
stage=migrated	Re-run install.sh — it will redo the upgrade and finish the outstanding steps
(file absent)	Re-run install.sh

A re-run at stage=migrated **reloads the database from the pre-upgrade dump**, replacing anything entered since it was made.

11.17 10.17 Check that a release is actually publishable

After adding or renaming anything under `lib/` or `bin/`, push it, then:

```
./tests/check-published.sh
```

That verifies every module is reachable over raw HTTP — which is what the `curl | bash` one-liners fetch. A half-published branch fails here rather than mid-run on a site.

Chapter 12

11. Troubleshooting

12.1 11.1 curl: (22) The requested URL returned error: 503

That is **raw.githubusercontent.com**, not your server and not the upgrade.

The raw host throttles and has short outages. When it answers 503 (or 429), `curl -f` gives up with exit 22 and the one-liner never reaches bash. Nothing has been changed on the machine at that point — the fetch failed before any work started, so it is always safe to run the command again.

Measured on 2026-09-12: three of five requests to the same raw URL returned 503, the rest 200, all served by the same edge. It comes and goes within seconds, which is exactly what retrying fixes.

Every command in this guide carries retry flags:

```
curl -fsSL --retry 8 --retry-max-time 180 <url> | bash
```

`--retry-delay` is deliberately **not** set: without it curl backs off exponentially (1s, 2s, 4s, 8s ...), which spreads the attempts across a minutes-long wobble instead of hammering the same dead edge for ten seconds.

Two things to know when reading the output:

- A `curl: (22) ... 503` line **followed by the script running** is a retry that succeeded. curl prints the error for each attempt; the run is fine.
- 503 repeated until the command dies means the raw host is down for longer than the retries cover. Wait a few minutes, or bypass the raw host entirely by running from a checkout, which fetches over git instead:

```
git clone https://github.com/Lesotho-eRegister-v1/upgrade-to-v1
cd upgrade-to-v1 && sudo ./catch-up.sh
```

Once a script is running it is largely past this: the entry points fetch `lib/` by shallow-cloning the repo **first** (github.com, not the raw host), and only fall back to per-file raw downloads. The fragile moment is the outer `curl ... | bash`.

12.2 11.2 fatal: detected dubious ownership in repository

Site clones end up owned by whoever created them. Since git 2.35 a repo owned by someone other than the current user is refused outright.

Every git call in the toolkit already passes `-c safe.directory='*'`, so this should not appear from the scripts themselves. If you see it running git by hand:

```
git -c safe.directory='*' -C /var/lib/v1/clinical-obs-forms status
```

Do **not** `chown -R` the clones as a fix. The scripts run as root and as the operator at different times; the ownership relaxation is per-invocation and deliberate.

12.3 11.3 A repo row says SKIP ... uncommitted local changes

– SKIP repo standard-config-ls uncommitted local changes – left untouched;
→ --force-repos to reset it onto Bokang-changes

This is working as designed. Sites hand-edit config, and silently discarding that would be the one destructive thing catch-up could do.

Decide which you want:

```
# see what changed
sudo git -C safe.directory='*' -C /var/lib/v1/standard-config-ls status
sudo git -C safe.directory='*' -C /var/lib/v1/standard-config-ls diff

# keep the changes: commit them, then re-run catch-up normally
# discard them:
sudo ./catch-up.sh --force-repos
```

The same row appears for a detached HEAD and for a repo on a different branch than the release pins. The detail text says which.

12.4 11.4 openmrddb:openmrs is not accepting connections right now

Expected straight after an upgrade or an EMR recreate. A freshly started v1 stack needs 30+ minutes before its database answers — hours on site hardware.

Nothing is wrong. The affected steps report a GAP and move on. Re-run catch-up.sh once the stack is up.

To check progress:

```
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard
sudo docker compose ps
sudo docker compose logs --tail 50 openmrs
```

12.5 11.5 The form import fails or does nothing

Symptom	Cause	F
not runnable (missing runner or credentials)	catch-up.sh has never completed step 2 on this host	F
not run – the EMR rejected the stored password	Password changed	Y
imported 0/48	Every form is already current	N
<form>.importErrors.txt written	Concepts unresolved	In
run failed – see /var/log/...	Usually the EMR still booting	V

```
sudo tail -n 60 /var/log/eregister-form-import.log
ls -l /var/lib/v1/form-import/
```

12.6 11.6 Forms deploy but stay unpublished

A form in **Draft** is deployed but not offered in the clinical app. The Implementer Interface's Import button always leaves one there, and so did this script before publishing was added.

```
sudo /usr/local/bin/bahmni-form-import.sh -k --publish-only \
-r /var/lib/v1/clinical-obs-forms
```

```
SELECT name, version FROM form WHERE published = 0 AND retired = 0 ORDER BY name;
```

If publishing itself is failing, the importer says which endpoint refused it:

Message	Meaning
publish endpoint answered HTTP 404 – trying the core form resource	bahmnicore is older than th
ERROR publishing: bahmniie said 404/405 and /form/<uuid> said HTTP 403	The account lacks the privile
ERROR: publish returned OK but '<name>' is still unpublished	The endpoint accepted the e
WARNING: cannot publish '<name>' – the server does not list it	The form name in the JSON

Run with `-v` to see the endpoint fallback decisions.

If a form keeps coming *back* published after you unpublish it by hand, that is the re-assertion working as designed — see §4.5. Use `--no-publish`.

12.7 11.7 Forms were retired but never came back

The retirement clears the recorded sha256 so the importer redeploys them. If that could not happen you would have seen:

```
[Δ] Could not rewrite /var/lib/v1/bahmni_form_import_state.json; the retired
[Δ] forms may be skipped as 'unchanged'. Force them with:
[Δ] sudo /usr/local/bin/bahmni-form-import.sh --force -r /var/lib/v1/clinical-obs-forms
```

Run exactly that. Then confirm:

```
SELECT name, version, retired FROM form
WHERE name LIKE '%2026%' ORDER BY name, version DESC;
```

You should see a new, un-retired version above the retired ones. If you need the old set back immediately, see §10.8.

12.8 11.8 The decode says still escaped after 5 pass(es)

The escaping nests deeper than the default cap:

```
sudo EREGISTER_FORM_DECODE_MAX_PASSES=12 ./catch-up.sh --decode
```

If it says the folder does not exist inside the container:

```
[x] /home/bahmni/clinical_forms does not exist in the 'openmrs' container – set
↪ EREGISTER_FORM_DECODE_DIR
```

Find the real path:

```
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard
sudo docker compose exec openmrs sh -c 'ls -ld /home/bahmni/* 2>/dev/null'
```

12.9 11.9 A nightly job is not producing files

```
✓ OK    cron    eregister-db-backup    systemd timer active (next: Mon 01:30)
✗ GAP   backup  newest dump              newest dump is 51h old
```

The timer exists but the runs are failing. Run it by hand and watch:

```
sudo /usr/local/bin/eregister-db-backup.sh
sudo tail -n 40 /var/log/eregister-db-backup.log
```

The most common cause is a compose service rename, so the runner cannot find openmrsdb. Check:

```
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard && sudo docker compose ps
```

A leftover `.part` file means a run was interrupted mid-write:

```
ls -l /var/lib/v1/db-backups/*.part
sudo rm /var/lib/v1/db-backups/*.part # then re-run the job
```

12.10 11.10 Changes were pulled but the site does not show them

Pulling a repo changes files on disk. It does not deploy anything. Which step is missing depends on what changed:

Changed	What still has to happen
standard-config-ls, openmrs-v1-modules, implementer-interface-release	The EMR must be recreated —
clinical-obs-forms	The form import must run, then
eregister_concepts_release_v1	The concept import must run, th
openmrs_reporting_release	catch-up.sh imports it, then th
bahmni-docker-ls	docker compose up -d — ca

If you ran catch-up with `--no-recreate`, none of the EMR-side changes are live yet:

```
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard
sudo docker compose up -d --force-recreate --renew-anon-volumes openmrs
```

12.11 11.11 The upgrade failed part-way

If the old stack had already been frozen, rollback ran automatically:

```
[△] Upgrade failed — initiating rollback to eRegister Lesotho 0.92.
[✓] Old stack restarted.
[△] Your database backup is preserved at: /var/lib/v1/bahmni-backup/openmrsdb_backup.sql
```

The backup is always preserved. If the automatic restart did not work:

```
cd ~/bahmni_docker && sudo docker compose start
```

Then read the error above the rollback lines — it names the file and line that failed.

12.12 11.12 install.sh says "already installed" but work is missing

```
cat /var/lib/v1/.eregister-upgrade-complete
```

stage=complete means the installer finished. If something is genuinely missing — a scheduled job, the forms, the report definitions — that is what catch-up.sh is for. It is almost never right to re-run install.sh with `--force`, because that reloads the database from the pre-upgrade dump.

12.13 11.13 Getting more detail out of a run

```
# every command, as it runs
sudo bash -x ./catch-up.sh --decode

# the importer, verbosely
sudo /usr/local/bin/bahmni-form-import.sh -k -v --dry-run \
    -r /var/lib/v1/clinical-obs-forms

# what a scheduled run did
sudo journalctl -u eregister-form-import.service -n 100
```

12.14 11.14 Where the logs are

```
/var/log/eregister-db-backup.log
/var/log/eregister-autopull.log
/var/log/eregister-form-import.log
/var/log/eregister-concept-import.log
```

Plus `journalctl -u <unit>` on a systemd host, and whatever the stack itself writes:

```
cd /var/lib/v1/bahmni-docker-ls/bahmni-standard  
sudo docker compose logs --tail 100 openmrs
```